PERSONALISED INTERVIEW PREP

Interview Revision Module

Technical skills Q&A, edge cases and rapid-fire rounds
+ non-technical and resume-based questions

Shreyansh Paliwal

B.Tech IT, GGSIPU (MAIT) | Full-stack, cloud and open-source

What is inside

Section 1 Technical: JavaScript, TypeScript, Node.js, Express, React, Next.js, Tailwind, Socket.IO, MongoDB, Redis, PostgreSQL, Git, Docker, CI/CD, AWS, Cloud Run, Python

Section 2 Non-technical: HR questions, deep-dives on every resume entry, open source, hackathons, leadership, number cheat sheet

Prepared October 2026. Framework-version details (Next.js, Tailwind, React) change often: re-check docs.

Contents

Section 1 — Technical Skills Revision 4

JavaScript 5
TypeScript 8
Node.js 10
Express.js and REST API Design 12
React.js 14
Next.js 16
Tailwind CSS 18
Socket.IO and WebSockets 19
MongoDB 21
Redis 23
PostgreSQL and SQL 25
Git 27
Docker 29
CI/CD and GitHub Actions 31
AWS (Lambda, DynamoDB, S3, EC2) 33
Google Cloud Run 35
Python and scikit-learn 37

Section 2 — Non-Technical and Resume-Based Questions 39

Introduction and HR Questions 40
Procurpal (Software Engineer Intern, Dec 2024 - Mar 2025) 42
MyAiTree (Software Engineer Intern, Jun - Aug 2024) 44
Alethea (Agentic AI log intelligence) 46
BehaveGuard (Behavioural authentication) 48
Open Source (Git, Augtoberfest, OpenNyAI) 50
Education, Hackathons and Leadership 52
Resume Number Cheat Sheet and Red-Flag Questions 54

How to use this module

- **Read in two passes.** Pass 1: read the questions and answer out loud before reading the model answer. Pass 2: skim the Edge cases and Rapid-fire boxes the night before.
- **Start with what your resume claims.** Every technology below is tagged with a "Link to your resume" note. Interviewers pivot from a skill to your claim ("you used Redis for rate limiting, show me how").
- **Answers marked (resume) or written as a "frame"** are templates built only from your resume text. Replace them with what you actually built, the real numbers and the real trade-offs. Do not repeat anything you cannot defend.
- **Section 1 = technical.** DSA, C/C++, OOP and System Design are intentionally excluded. **Section 2 = non-technical and resume deep-dives.**
- **Version-sensitive facts** (Next.js caching, Tailwind v4, React 19, Express 5, AWS limits) change. Check the official docs once before the interview.
- **Suggested schedule:** one topic per day for 3 weeks, then a rapid-fire-only pass for the last 2 days.

Legend

Q1.	Question and model answer
Edge cases	Gotchas and trick questions interviewers love
Rapid-fire	One-line answers to memorise

Section 1 — Technical Skills Revision

Every technology on your resume (except DSA, C/C++, OOP and System Design). Each topic has: top interview questions with model answers, edge cases and gotchas, and a rapid-fire round. Topics tied to your projects or internships carry a "Link to your resume" note, because interviewers will pivot from the skill to your claim.

JavaScript

> Link to your resume: used in every internship and project (React, Node, Next.js). Expect output-prediction questions and event-loop questions for sure.

Q1. Explain var, let and const. What are hoisting and the Temporal Dead Zone?

`var` is function-scoped, hoisted and initialised to `undefined`. `let` and `const` are block-scoped and hoisted too, but they stay uninitialised until their declaration line runs. That window is the **Temporal Dead Zone (TDZ)**, and accessing the variable there throws a `ReferenceError`.

- `const` prevents re-assignment of the binding, not mutation of the object it points to.
- Function declarations are hoisted with their body; function expressions (`const f = () => {}`) are not callable before the line runs.
- A top-level `var` becomes a property of `window` (browser); `let`/`const` do not.

Q2. What is a closure? Give a practical use and a classic pitfall.

A closure is a function bundled with references to the variables of the scope where it was created, so it keeps access to them even after the outer function has returned. Uses: data privacy (module pattern), factories, memoization, debounce/throttle, event handlers, React hooks.

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0); // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0); // 0 1 2
```

`var` shares one binding across iterations; `let` creates a fresh binding per iteration. **React link:** stale closures in `useEffect` or `setInterval` capture old state values.

Q3. Explain the event loop. What is the difference between microtasks and macrotasks?

JavaScript runs on one thread with a call stack. Async work (timers, I/O, network) is handled by the host (browser or Node) and its callbacks are queued. When the stack is empty, the event loop first drains the **microtask queue** (promise `.then`/`catch`/`finally`, `await` continuations, `queueMicrotask`, `MutationObserver`), then takes one **macrotask** (`setTimeout`, `setInterval`, I/O, UI events), and repeats. Microtasks scheduled by microtasks run before the next macrotask, so a recursive microtask can starve the loop.

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// 1 4 3 2
```

Q4. How does `this` work? What do call, apply and bind do?

`this` is decided by **how a function is called**: method call gives the object before the dot; plain call gives `undefined` in strict mode (or global object in sloppy mode); `new` gives the new object; `call`/`apply`/`bind` set it explicitly. **Arrow functions** have no own `this`; they capture it lexically from the enclosing scope, so they cannot be rebound and cannot be used as constructors.

- `fn.call(obj, a, b)` invokes immediately with arguments listed.
- `fn.apply(obj, [a, b])` invokes immediately with an array.
- `fn.bind(obj, a)` returns a new function with `this` (and optionally leading arguments) fixed.

Q5. How does prototypal inheritance work? Are ES6 classes real classes?

Every object has an internal `[[Prototype]]` link. Property lookup walks this chain until found or until `null`. `class` is syntactic sugar over constructor functions and prototypes: methods live on `Class.prototype`, and `extends` wires the prototype chain. `Object.create(proto)` creates an object with a chosen prototype. `hasOwnProperty` / `Object.hasOwn` distinguish own from inherited properties.

Q6. Promises vs async/await. Explain Promise.all, allSettled, race and any.

`async/await` is syntax over promises: an `async` function always returns a promise, and `await` pauses that function (not the thread) until the promise settles, resuming as a microtask.

- `Promise.all`: resolves with all results, rejects immediately on the first rejection.
- `Promise.allSettled`: waits for all, never rejects, gives `{status, value|reason}` per item.
- `Promise.race`: settles with the first settled promise (resolve or reject).
- `Promise.any`: resolves with the first fulfilled one; rejects with `AggregateError` only if all reject.

Sequential vs parallel: `await` inside a `for` loop runs one after another; `await Promise.all(items.map(fn))` runs them concurrently.

Q7. == vs ===. How does type coercion work?

`===` compares value and type with no coercion. `==` coerces first (e.g. `"5" == 5` is true, `null == undefined` is true, `null == 0` is false). Rule of thumb: always use `===`, except `x == null` as a deliberate check for both `null` and `undefined`. `Object.is` differs from `===` only for `NaN` (equal to itself) and `+0/-0` (different).

Q8. Implement debounce. How is it different from throttle?

Debounce delays execution until the caller has been quiet for N ms (search box input). Throttle guarantees at most one execution per N ms (scroll, resize).

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

Q9. Shallow copy vs deep copy. How do you deep clone?

Spread (`{...obj}`), `Object.assign` and `Array.slice` copy only the first level; nested objects stay shared. Deep clone options: `structuredClone(obj)` (modern, handles Date, Map, Set, cycles, but not functions or class prototypes), `JSON.parse(JSON.stringify(obj))` (loses `undefined`, functions, Dates become strings, fails on cycles, BigInt throws), or a library like lodash `cloneDeep`.

Q10. map, filter, reduce, forEach. Why is `await` inside forEach a bug?

`map` transforms, `filter` selects, `reduce` folds to one value, `forEach` returns nothing. `forEach` does not wait for async callbacks, so code after it runs before the work finishes. Use `for...of` with `await` for sequential, or `await Promise.all(arr.map(async ...))` for parallel.

Q11. CommonJS vs ES Modules.

CommonJS: `require` / `module.exports`, synchronous, dynamic, value copies at load time, default in older Node. ESM: `import` / `export`, static (enables tree-shaking), asynchronous loading, **live bindings**, top-level `await`, strict mode by default. In Node, ESM needs `"type": "module"` or `.mjs`; `__dirname` is not available in ESM (use `import.meta.url`).

Q12. Event bubbling, capturing and delegation.

Events travel capture (window down to target), target, then bubble (target up to window). `addEventListener(type, fn, true)` listens in capture phase. **Delegation** attaches one listener to a parent and uses `event.target` to handle many children, which saves memory and works for dynamically added elements. `stopPropagation` stops travel; `preventDefault` stops the browser's default action; they are different things.

Q13. localStorage vs sessionStorage vs cookies.

localStorage: about 5 MB, persists until cleared, same-origin, not sent to the server. sessionStorage: per tab, cleared when the tab closes. Cookies: about 4 KB, sent with every request, can be `HttpOnly` (not readable by JS, protects against XSS theft), `Secure`, `SameSite`. Store session tokens in `HttpOnly` cookies rather than localStorage.

Q14. What causes memory leaks in JavaScript and how do you find them?

Common causes: forgotten timers/intervals, event listeners never removed, detached DOM nodes still referenced, growing global caches, closures holding large objects. Find them with Chrome DevTools Memory tab (heap snapshots comparison, allocation timeline). In React, clean up in the `useEffect` return function.

Edge cases and gotchas

- `typeof null === "object"` (historic bug); `typeof NaN === "number"`; `NaN !== NaN`, use `Number.isNaN`.
- `0.1 + 0.2 !== 0.3` (floating point); compare with an epsilon or use integers.
- `[1, 2, 10].sort()` gives `[1, 10, 2]`; default sort is lexicographic. Use `(a, b) => a - b`.
- `['1','2','3'].map(parseInt)` gives `[1, NaN, NaN]` because `map` passes the index as the radix.
- `||` treats `0`, `""`, `false` as missing; `??` only treats `null`/`undefined`. `a?.b` short-circuits safely.
- `[] + {}` is `"[object Object]"`; `[] == ![]` is `true`.
- Objects as keys: `obj[{}]` becomes `"[object Object]"`; use `Map` for arbitrary keys; `Map` keeps insertion order and has O(1) size.
- Mutating an array while iterating it with `forEach`/`for...in` gives surprising results; `for...in` also walks inherited keys and is the wrong tool for arrays.
- `async` function that throws returns a rejected promise; it does not throw synchronously to the caller.
- `Array(3)` creates holes; `map` skips holes.

Rapid-fire round

Question	Answer
Output of `console.log(typeof undefined, typeof null, typeof [])`	`undefined object object`
What does `Promise.resolve().then(f)` do with `f`	Schedules it as a microtask
Why `const arr = []; arr.push(1)` works	const protects the binding, not the contents
Difference between `null` and `undefined`	undefined = not assigned; null = deliberate empty value
What is a pure function	Same input gives same output, no side effects
What is `Symbol` for	Unique, non-colliding property keys
Hoisting of classes	Hoisted but in TDZ, same as let/const
`Object.freeze` depth	Shallow only
Is JS pass by value or reference	Always by value; for objects the value is a reference
Purpose of `WeakMap`	Keys held weakly, so entries can be garbage collected

TypeScript

> Link to your resume: Procurpal backend (TypeScript, Cron Jobs, DynamoDB) and Alethea. Be ready to say how types helped you in the Qsign integration.

Q1. Why TypeScript over JavaScript? What does it not do?

Static typing catches errors at compile time, gives autocomplete and safe refactoring, and documents contracts (especially API/DB shapes). It does **not** validate anything at runtime: types are erased during compilation, so data from `JSON.parse`, HTTP bodies or the DB must still be validated (zod, io-ts, class-validator).

Q2. interface vs type.

Both describe object shapes. `interface` supports declaration merging and `extends`, and is the usual choice for public object contracts. `type` can alias anything: unions, intersections, tuples, primitives, mapped and conditional types. Rule of thumb: `interface` for objects/classes, `type` for unions and computed types.

Q3. any vs unknown vs never.

`any` turns off type checking (it spreads silently). `unknown` is the type-safe top type: you must narrow it (`typeof`, `instanceof`, type guard) before use. `never` is the type of values that cannot occur: functions that always throw, and the leftover branch in an exhaustive `switch`, which you can use for compile-time exhaustiveness checks.

Q4. Explain generics with an example.

Generics let one function or type work over many types while preserving relationships between them.

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }
async function fetchJson<T>(url: string): Promise<T> {
  const res = await fetch(url);
  return res.json() as Promise<T>;
}
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; }
```

Constraints (`extends`) limit what T can be; defaults (`<T = string>`) give fallback types.

Q5. Name the most useful utility types.

`Partial<T>` (all optional), `Required<T>`, `Readonly<T>`, `Pick<T, K>`, `Omit<T, K>`, `Record<K, V>`, `Exclude<U, X>`, `Extract<U, X>`, `NonNullable<T>`, `ReturnType<F>`, `Parameters<F>`, `Awaited<T>`. They are built from mapped and conditional types.

Q6. Union and intersection types; narrowing; discriminated unions.

`A | B` is either; `A & B` has all members of both. Narrowing refines a union via `typeof`, `instanceof`, `in`, equality, truthiness, or user-defined guards (`x is Foo`). A **discriminated union** adds a literal tag so `switch` narrows safely.

```
type Result = { ok: true; data: string } | { ok: false; error: Error };
function handle(r: Result) {
  if (r.ok) return r.data; else throw r.error;
}
```

Q7. Enums vs union of string literals.

Enums emit runtime code (numeric enums also have reverse mapping), `const enum` is inlined but problematic with some tooling. A union like `"idle" | "loading" | "done"` has zero runtime cost, works naturally with JSON, and is generally preferred. Use `as const` objects when you also want runtime values.

Q8. What is structural typing?

TypeScript compares types by shape, not by name ("duck typing"). Any object with the required properties satisfies the type. That is why two differently named interfaces with the same shape are interchangeable.

Q9. `as` assertion vs `satisfies` vs non-null `!`.

`as T` tells the compiler to trust you (no runtime check; can hide bugs). `satisfies T` checks that a value conforms to T while keeping its narrower inferred type. `x!` removes `null`/`undefined` from the type, again with no runtime guarantee. Prefer narrowing and guards.

Q10. Which tsconfig options matter?

`strict` (enables `strictNullChecks`, `noImplicitAny`, `strictFunctionTypes`, etc.), `target`/`module`/`moduleResolution`, `esModuleInterop`, `skipLibCheck`, `paths`, `noUncheckedIndexedAccess`, `outDir`, `sourceMap`. Mention that `strict: true` is the baseline for any serious project.

Q11. keyof, typeof, indexed access, mapped and conditional types.

`keyof T` gives the union of keys; `typeof value` in a type position takes the type of a value; `T["a"]` indexes a type. Mapped type: `{ [K in keyof T]?: T[K] }`. Conditional: `T extends U ? X : Y`, with `infer` to extract types (`T extends Promise<infer R> ? R : T`).

Edge cases and gotchas

- Excess property checks apply only to fresh object literals, not to variables.
- `JSON.parse` and `res.json()` return `any`: a silent hole in an otherwise typed codebase.
- `Object.keys(obj)` returns `string[]`, not `(keyof T)[]`, by design (objects may have extra keys).
- Array index access is typed as `T`, not `T | undefined`, unless `noUncheckedIndexedAccess` is on.
- Function parameter bivariance for methods vs strictness for function-typed properties.
- Type-only imports (`import type`) are erased and avoid circular import issues.
- Declaration files (`.d.ts`) and `@types/*` (DefinitelyTyped) supply types for JS libraries.

Rapid-fire round

Question	Answer
Does TS code run in the browser/Node directly	No, compile to JS first (tsc, esbuild, swc) or use a runner like tsx
`readonly` vs `const`	readonly is for properties and is compile-time only; const is for bindings
Optional vs `| undefined`	`a?: T` may be omitted; `a: T | undefined` must be present
What is `void` vs `undefined`	void is for ignored return values of functions
Tuple type	Fixed-length array with positional types, e.g. `[string, number]`
Abstract class vs interface	Abstract class can hold implementation and exists at runtime; interface is erased

Node.js

> Link to your resume: Node.js is the core of your backend work at Procurpal, Alethea, and your API-heavy projects.

Q1. How does Node.js handle concurrency if it is single-threaded?

JavaScript code runs on one main thread, but Node uses **libuv** with a thread pool (default 4 threads) and OS async primitives (epoll/kqueue/IOCP) for I/O, DNS, file system and crypto work. While I/O is pending, the main thread serves other requests, which makes Node excellent for I/O-bound workloads and poor for CPU-bound ones.

Q2. Explain the Node event loop phases.

Order per iteration: **timers** (`setTimeout`/`setInterval` callbacks), **pending callbacks**, idle/prepare, **poll** (retrieve new I/O events, run I/O callbacks), **check** (`setImmediate`), **close callbacks**. Between each callback, Node drains `process.nextTick` first, then promise microtasks.

Q3. process.nextTick vs setImmediate vs setTimeout(fn, 0).

`process.nextTick` runs right after the current operation, before promise microtasks and before the loop continues, and can starve I/O if abused. `setImmediate` runs in the check phase, after the poll phase. `setTimeout(0)` runs in the timers phase (minimum about 1 ms). From the main module their order is non-deterministic; inside an I/O callback `setImmediate` always fires first.

Q4. How do you handle CPU-intensive tasks in Node?

Offload: `worker_threads` (shared memory, good for compute), `child_process` (separate process), `cluster` (multiple processes to use all cores for request handling), a job queue (BullMQ/SQS) with separate workers, or a different service/Lambda. Never block the loop with sync crypto, huge `JSON.parse`, or heavy loops.

Q5. What are streams and backpressure?

Streams process data in chunks (Readable, Writable, Duplex, Transform) instead of loading everything into memory. **Backpressure** is the mechanism by which a slow consumer signals a fast producer to pause (`write()` returns `false`, wait for `drain`). Use `pipeline` from `stream/promises` to wire streams with correct error handling and cleanup.

Q6. Cluster vs worker_threads vs child_process.

Cluster forks processes that share a server port (separate memory, good for scaling HTTP). worker_threads run JS in threads inside one process with message passing and optional SharedArrayBuffer. child_process spawns any external program or Node script.

Q7. How do you handle errors in Node?

Use try/catch with async/await; always handle promise rejections; Express error middleware for HTTP; custom error classes with status codes. Listen to `unhandledRejection` and `uncaughtException` for logging only, then exit and let the process manager (PM2, Docker restart policy, ECS/Cloud Run) restart a clean process. Since Node 15 an unhandled rejection crashes the process by default.

Q8. How do you do graceful shutdown?

On `SIGTERM`/`SIGINT`: stop accepting new connections (`server.close`), finish in-flight requests, close DB/Redis connections, then `process.exit`. Containers get SIGTERM then SIGKILL after a grace period, so set a timeout. This matters on Cloud Run and in Docker.

Q9. EventEmitter and Buffer in one line each.

EventEmitter is the pub/sub base class (`on`, `once`, `emit`, `off`); too many listeners triggers a leak warning. Buffer is a fixed-size chunk of raw binary memory (used for files, sockets, crypto); `Buffer.from`, `toString('base64')`.

Q10. How would you implement a cron job in a Node backend? What can go wrong? (your Procurpal resume line)

Options: in-process scheduler (`node-cron`, `agenda`, BullMQ repeatable jobs), OS cron, or managed schedulers (AWS EventBridge Scheduler triggering a Lambda). Issues to discuss: **overlapping runs** (job takes longer than the interval, so use a lock or skip-if-running), **multiple instances** running the same job (use a distributed lock in Redis/DynamoDB conditional write, or one dedicated scheduler), **idempotency** (safe to rerun), retries with backoff, timeouts, time zones/DST, monitoring and alerting for silent failures, and no scheduling inside a serverless function that can scale to zero.

Q11. Common Node performance and memory tips.

Avoid sync APIs on the request path, reuse connections (HTTP keep-alive, DB pools), cache hot data (Redis), stream big payloads, enable gzip/brotli, use `--inspect`/clinic.js/heap snapshots, set `--max-old-space-size` deliberately, limit payload sizes.

Edge cases and gotchas

- `require` caches modules: a module executes once; mutable exports are shared.
- Circular `require` dependencies return partially filled `exports`.
- `fs.readFileSync` in a request handler blocks every user.
- `JSON.parse` of untrusted giant input can freeze the process (limit body size).
- Unclosed DB connections or listeners in a loop cause leaks; each Lambda invocation reusing a warm container may keep globals.
- In ESM, `__dirname` and `require` do not exist.
- Environment variables are strings; `"false"` is truthy.

Rapid-fire round

Question	Answer
Is Node multi-threaded	JS execution is single-threaded; libuv and worker threads add parallelism
What is `package-lock.json` for	Reproducible installs with exact dependency versions
dependencies vs devDependencies	Runtime vs build/test-only packages
`npm ci` vs `npm install`	ci installs exactly from lockfile, clean, for CI/CD
What is middleware	A function in the request/response pipeline with access to req, res, next
What is libuv	C library providing event loop, thread pool, async I/O

Express.js and REST API Design

> Link to your resume: you built RESTful APIs for the Qsign integration and run Express in your stack. Expect questions on middleware, auth, status codes, idempotency and webhooks.

Q1. What is middleware and why does order matter?

Middleware are functions `(req, res, next)` run in the order registered. They can modify `req`/`res`, end the response, or call `next()` to continue. Registering the body parser after the routes means `req.body` is undefined in handlers; registering the 404 handler before routes swallows everything. Typical order: security headers (helmet), CORS, logging, body parsing, auth, routes, 404, error handler.

Q2. How does error handling work in Express?

Error middleware has four parameters `(err, req, res, next)` and is registered last. Synchronous throws and `next(err)` reach it. In **Express 4**, rejected promises in async handlers are not caught automatically (wrap with a helper or `express-async-errors`); **Express 5** forwards rejected promises to the error handler.

Q3. PUT vs PATCH vs POST. What does idempotent mean?

POST creates (not idempotent), PUT replaces a whole resource (idempotent), PATCH partially updates (usually not guaranteed idempotent), DELETE removes (idempotent), GET/HEAD are safe and idempotent. Idempotent means repeating the request leaves the same final state. For non-idempotent operations like payments or e-signing requests, accept an **Idempotency-Key** header and store results so retries don't duplicate.

Q4. Important HTTP status codes.

200 OK, 201 Created, 204 No Content, 301/302 redirects, 304 Not Modified, 400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but not allowed), 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout.

Q5. Sessions vs JWT authentication.

Sessions: server stores state (in Redis/DB), client holds an opaque session ID in a cookie; easy to revoke, needs shared store when scaled. JWT: signed, self-contained token (header.payload.signature), stateless verification, hard to revoke before expiry (use short expiry + refresh tokens, or a denylist). A JWT is signed, not encrypted, so never put secrets in the payload. Verify the algorithm explicitly (reject `alg: none`), check `exp`, `iss`, `aud`.

Q6. How do you secure an Express API?

helmet (headers), CORS with explicit origins, rate limiting (express-rate-limit with Redis store), input validation (zod/joi), parameterised queries/ORM (SQL injection), sanitising NoSQL operators (`$ne`, `$gt` injection), bcrypt/argon2 for passwords, HTTPS, `HttpOnly`+`Secure`+`SameSite` cookies, CSRF protection for cookie auth, secrets in env/Secret Manager, dependency audit (`npm audit`), body size limits, least-privilege DB/IAM users.

Q7. What is CORS and how does preflight work?

Browsers block cross-origin requests unless the server allows them via `Access-Control-Allow-Origin` and related headers. For non-simple requests (custom headers, `PUT`/`DELETE`, JSON content type) the browser first sends an `OPTIONS` preflight; the server must respond with allowed methods/headers. CORS is enforced by browsers only; it is not server-to-server security.

Q8. How do you design pagination, filtering and versioning?

Offset pagination (`?page=&limit=`) is simple but slow and unstable for large or changing data; cursor/keyset pagination (`?after=<id>`) is stable and fast. Versioning: URL (`/v1/`) or header. Return consistent error shapes, correct status codes, and rate-limit headers.

Q9. How would you integrate a third-party API like Qsign reliably? (resume)

Wrap the client in one module with timeouts, retries with exponential backoff and jitter (only for idempotent or keyed calls), and circuit breaking. Persist the external request/status in your DB so you can reconcile. Prefer **webhooks** for status updates (verify signature, respond 2xx fast, process asynchronously, dedupe by event ID) with a **cron reconciliation** as a safety net for lost webhooks. Log correlation IDs and never log secrets or documents.

Q10. What is a webhook and how do you secure one?

A webhook is an HTTP callback the provider calls when an event occurs. Secure it with a shared-secret HMAC signature (compare with timing-safe equality), timestamp checks to prevent replay, HTTPS, IP allowlist if offered, and idempotent handlers because providers retry.

Edge cases and gotchas

- 401 vs 403 confusion is a classic question.
- `app.use(express.json())` must come before routes; large bodies need a limit.
- Trust proxy: behind a load balancer set `app.set('trust proxy', ...)` or rate limiting and IP logging see the proxy IP.
- Sending a response twice throws "Cannot set headers after they are sent".
- `res.send` after an error is already handled; always `return` after sending.
- Cookies with `SameSite=None` require `Secure`.
- Offset pagination with concurrent inserts duplicates or skips rows.

Rapid-fire round

Question	Answer
Express vs Koa vs Fastify	Express minimal and ubiquitous; Fastify faster with schema validation; Koa uses async middleware
REST constraints	Client-server, stateless, cacheable, uniform interface, layered system
Difference between authentication and authorisation	Who you are vs what you may do
What is `req.params` vs `req.query` vs `req.body`	Path segments, query string, payload
Why not store JWT in localStorage	XSS can read it; prefer HttpOnly cookie
What is rate limiting for	Abuse and DoS protection, fair usage

React.js

> Link to your resume: Procurpal real-time e-Auction (React, WebSockets, Tailwind, 20+ responsive screens), Alethea (Next.js dashboard). Expect hooks, performance, and real-time state questions.

Q1. What is the Virtual DOM and how does reconciliation work?

React keeps a lightweight tree of elements in memory. On state change it renders a new tree, **diffs** it against the previous one, and applies only the minimal DOM changes. Heuristics: different element types produce a rebuild of that subtree; same type updates attributes; lists are matched by `key`. React 16+ uses the **Fiber** architecture, which makes rendering interruptible and prioritised.

Q2. Why are keys important in lists? Why not use the index?

Keys let React identify which items changed, moved, or were removed. With index keys, inserting or reordering items makes React reuse the wrong component instance, causing wrong state in inputs and unnecessary re-renders. Use a stable unique ID from the data.

Q3. Explain useState, and why updates are "asynchronous".

`setState` schedules a re-render; the variable in the current render does not change immediately. React 18 **batches** updates automatically (even in promises and timeouts). Use the functional form `setCount(c => c + 1)` when the new state depends on the previous one.

Q4. useEffect: dependency array and cleanup.

Runs after render. No array: after every render. `[]`: once on mount (and unmount cleanup). `[a, b]`: when a or b change. The returned function cleans up before the next run and on unmount: clear timers, abort fetch, **close WebSocket**, remove listeners. Missing dependencies cause stale closures; the ESLint rule `react-hooks/exhaustive-deps` helps.

Q5. useMemo vs useCallback vs React.memo.

`React.memo(Component)` skips re-render if props are shallow-equal. `useMemo` caches a computed value; `useCallback` caches a function identity (so memoised children don't re-render because of a new function prop). Don't use them everywhere: measure first, because the memo check also costs.

Q6. useRef vs useState.

`useRef` holds a mutable value that persists across renders and does **not** trigger a re-render when changed. Use for DOM nodes, timers, previous values, and the latest socket instance. `useState` triggers a re-render.

Q7. useReducer vs useState; Context API drawbacks.

`useReducer` suits complex state with many transitions (and testable pure reducers). Context passes data without prop drilling but **every consumer re-renders when the value changes**, so split contexts, memoise the value, or use a state library (Redux Toolkit, Zustand) for frequently changing state.

Q8. Controlled vs uncontrolled components.

Controlled: React state is the single source of truth via `value` + `onChange`. Uncontrolled: DOM holds the value, read through a `ref` (or `FormData`). Libraries like React Hook Form use uncontrolled inputs for performance.

Q9. What are custom hooks and the rules of hooks?

Custom hooks (functions starting with `use`) extract reusable stateful logic (e.g. `useWebSocket`, `useDebounce`). Rules: call hooks only at the top level (not in loops/conditions) and only from components or other hooks, because React tracks hooks by call order.

Q10. How do you optimise a slow React app?

Profile with React DevTools Profiler. Then: avoid unnecessary renders (memo, split components, lift state down), virtualise long lists (react-window), code-split (`React.lazy` + `Suspense`), debounce inputs, memoise expensive computations, optimise images, avoid creating new objects/functions in props to memoised children, use `useTransition`/`useDeferredValue` for non-urgent updates.

Q11. What did React 18 and 19 add?

React 18: concurrent rendering, automatic batching, `useTransition`, `useDeferredValue`, `useId`, streaming SSR with Suspense. React 19: Actions and `useActionState`, `useOptimistic`, `use()` for reading promises/context, `ref` as a prop (no `forwardRef` needed), document metadata support, and Server Components stable in frameworks.

Q12. How would you build the real-time e-Auction UI? (resume)

Open one socket per auction page in a `useEffect` (cleanup on unmount), keep the connection in a `ref` or a context/provider, and update a normalised store (reducer) on `bid:new` events. The **server is the source of truth** for the highest bid and the end time; the client only displays and sends bid intents. Handle reconnect by re-fetching a snapshot, show optimistic UI carefully (roll back on rejection), sync clocks using server time for the countdown, and throttle UI updates when events burst. Responsive screens: mobile-first Tailwind breakpoints, shared component library.

Q13. What are error boundaries?

Class components (or libraries like `react-error-boundary`) that catch render-time errors in children and show fallback UI. They do not catch errors in event handlers, async code, or SSR.

Edge cases and gotchas

- StrictMode (dev) double-invokes renders and effects to expose impure code and missing cleanup; not a bug.
- Mutating state directly (`arr.push`) does not trigger a re-render because reference equality is used.
- `useEffect` with an object/array in deps re-runs every render since a new reference is created each time.
- Stale closure example: `setInterval` inside an effect with `[]` deps always sees initial state; fix with functional update or a ref.
- Setting state in render causes an infinite loop.
- `key` change forces a full remount (useful trick to reset component state).
- Fetching in `useEffect` can race: ignore stale responses with an abort flag/`AbortController`.
- `useLayoutEffect` runs synchronously before paint; it warns on the server.

Rapid-fire round

Question	Answer
Props vs state	Props come from parent and are read-only; state is owned and changeable
What triggers a re-render	State change, props change, parent re-render, context change
Fragment	Group children without extra DOM node
Lifting state up	Move shared state to the closest common parent
Portal	Render children into a DOM node outside the parent hierarchy (modals)
SSR vs CSR	HTML generated on server vs in the browser
Hydration	Attaching React event handlers to server-rendered HTML

Next.js

> Link to your resume: Alethea (Next.js, OAuth GitHub). Be sure which router (App or Pages) and which version you used.

Q1. What are CSR, SSR, SSG and ISR?

CSR renders in the browser after JS loads. SSR renders HTML per request on the server (fresh data, slower TTFB). SSG pre-renders at build time (fastest, static). ISR re-generates static pages in the background after a revalidate window (or on demand) so you get static speed with periodically fresh data.

Q2. App Router vs Pages Router.

Pages Router uses `pages/` with `getServerSideProps`/`getStaticProps`. App Router (`app/`) uses nested layouts, React Server Components by default, streaming with `loading.js` and Suspense, `error.js`, route handlers (`route.js`), and Server Actions. New projects should use the App Router.

Q3. Server Components vs Client Components.

Server Components run only on the server: they can access databases/secrets directly, add no JS to the client bundle, and cannot use state, effects or browser APIs. Add `"use client"` to opt into Client Components for interactivity. Props crossing the boundary must be serialisable. Keep client components as leaves of the tree.

Q4. How does data fetching and caching work?

Server Components can `await fetch()` or query the DB directly. Caching defaults and APIs have **changed between versions** (for example, Next 15 stopped caching `fetch` and GET route handlers by default, and newer versions add explicit caching directives). Say which version you used and describe `revalidate`, tags (`revalidateTag`) and `revalidatePath`. Check the official docs for the current model before the interview.

Q5. What are Server Actions?

Async functions marked `"use server"` that run on the server and can be called from forms or client components, replacing hand-written API routes for mutations. They are public endpoints in effect, so validate input and check authorisation inside them.

Q6. Middleware in Next.js.

Runs before a request completes: redirects, rewrites, auth checks, headers, A/B tests. It runs at the edge runtime with limited Node APIs, so keep it light. Recent versions are renaming/reworking this layer, so confirm naming for your version.

Q7. How do you implement GitHub OAuth in Next.js? (resume)

Use Auth.js (NextAuth) with the GitHub provider or implement the flow manually: redirect to GitHub authorize URL with `client_id`, `scope` and a random `state` (CSRF protection); GitHub redirects back with a `code`; the server exchanges it for an access token using the client secret; fetch the user profile; create a session (HttpOnly cookie or JWT). Store tokens server-side, request minimum scopes, never expose the client secret to the browser.

Q8. next/image, next/link, next/font, dynamic routes.

`next/image` gives responsive sizes, lazy loading, modern formats and layout-shift prevention. `next/link` prefetches routes. `next/font` self-hosts fonts to avoid layout shift. Dynamic routes: `[id]`, catch-all `[...slug]`, optional catch-all `[[...slug]]`.

Q9. Environment variables.

Server-only by default; only variables prefixed `NEXT_PUBLIC_` are inlined into the browser bundle at **build time**. Never put secrets in `NEXT_PUBLIC_` variables.

Edge cases and gotchas

- Hydration mismatch: server and client HTML differ (e.g. `Date.now()`, `window` checks, random IDs); fix with `useEffect` for client-only values or `suppressHydrationWarning`.
- Using `useState` or `onClick` in a Server Component is a build error; add `"use client"`.
- `NEXT_PUBLIC_` values are baked at build time, so changing them at runtime in a Docker image does nothing.
- Dynamic functions (`cookies()`, `headers()`) opt a route into dynamic rendering.
- Serverless deployment: each route may cold start; avoid heavy module-level init.
- Docker: use `output: "standalone"` to shrink the image.

Rapid-fire round

Question	Answer
Why Next.js over plain React	Routing, SSR/SSG, SEO, API routes, optimisation built in
`getStaticProps` vs `getServerSideProps`	Build time vs every request (Pages Router)
What is streaming	Sending HTML in chunks as parts become ready
Edge runtime	Lightweight V8 runtime near users, limited Node APIs
Why is SEO better with SSR	Crawlers get full HTML immediately

Tailwind CSS

> Link to your resume: used for the 20+ responsive screens of the e-Auction module.

Q1. What is utility-first CSS and what are the trade-offs?

Compose styles from small single-purpose classes in markup (`flex items-center p-4`). Pros: no naming, no context switching, tiny production CSS (only used classes), consistent design tokens, fast iteration. Cons: verbose markup, a learning curve, needs component abstraction to avoid repetition.

Q2. How does responsive design work?

Mobile-first: unprefixed utilities apply to all sizes; prefixes like `sm:`, `md:`, `lg:`, `xl:` apply at that min-width and up. Example: `grid grid-cols-1 md:grid-cols-3`. Also `max-*` variants and container queries (`@container`).

Q3. How are classes generated and why do dynamic class names break?

Tailwind scans your source for complete class-name strings and generates only those. `` `text-${color}-500` `` is never seen as a full string, so the CSS is missing. Use complete names in a lookup map: `{ red: "text-red-500" }`.

Q4. Dark mode, customisation and reuse.

`dark:` variant (media-based or class/selector-based). Customise colours, spacing and fonts in the theme (`tailwind.config` in v3; CSS-first `@theme` in v4). Reuse through components (React), not `@apply` everywhere; `clsx`/`tailwind-merge` to combine conditional classes safely.

Q5. What changed in Tailwind v4?

CSS-first configuration with `@import "tailwindcss"` and `@theme`, a faster engine, automatic content detection, and modern CSS features (cascade layers, container queries). Know which version your project used.

Edge cases and gotchas

- Class order in markup does not control precedence; CSS order in the generated stylesheet does. Use `tailwind-merge` for conflicting classes.
- `group` and `peer` variants style children/siblings based on parent/sibling state.
- Arbitrary values: `w-[37px]`, `bg-[#1da1f2]`.
- `flex` vs `grid`: flex for one-dimensional layout, grid for two-dimensional.
- Important modifier: `!p-4`.

Rapid-fire round

Question	Answer
Breakpoint approach	Mobile-first, min-width
How to remove unused CSS	Automatic via content scanning
`space-x-4` vs `gap-4`	Margin-based between siblings vs flex/grid gap
`sr-only`	Visually hidden but available to screen readers

Socket.IO and WebSockets

> Link to your resume: real-time e-Auction module (React, WebSockets). This is a rich area for design follow-ups.

Q1. WebSocket vs HTTP vs SSE vs long polling.

HTTP is request/response. Long polling simulates push by holding requests open. Server-Sent Events: one-way server to client over HTTP, automatic reconnect, simple. WebSocket: persistent, full-duplex, low-overhead TCP connection after an HTTP `Upgrade` handshake; ideal for bi-directional real-time (bidding, chat, collaboration).

Q2. Socket.IO vs raw WebSocket.

Socket.IO is a library on top of (not a pure implementation of) WebSocket: automatic reconnection, heartbeats, rooms and namespaces, acknowledgements, broadcast helpers, fallback to HTTP long-polling, and middleware. Cost: a custom protocol, so a plain WebSocket client cannot connect to a Socket.IO server.

Q3. Rooms and namespaces.

A namespace (`/auction`) is a separate communication channel on one connection. A room is a server-side grouping of sockets inside a namespace: `socket.join("auction:42")`, `io.to("auction:42").emit("bid:new", data)`. One room per auction is the natural fit.

Q4. How do you scale Socket.IO across multiple servers?

Use **sticky sessions** at the load balancer (needed for the HTTP long-polling handshake) and the **Redis adapter** (or another adapter) so broadcasts reach sockets connected to other nodes. Each node keeps its own connections; the adapter relays events through pub/sub.

Q5. How do you authenticate sockets?

Validate a token during the handshake (`io.use((socket, next) => ...` reading `socket.handshake.auth.token`), attach the user to `socket.data`, and check permissions per event. Don't trust identifiers sent in event payloads.

Q6. Design the bidding flow so two simultaneous bids don't corrupt the auction. (resume deep-dive)

Treat the server as the single authority. A bid event arrives, the server validates (auction open, amount > current highest + minimum increment, user eligible) and updates the highest bid **atomically**: a DB conditional update / compare-and-set, a Redis Lua script or `WATCH`/`MULTI`, or a DynamoDB conditional write (`ConditionExpression: amount < :newBid`). Only after the write succeeds, broadcast `bid:new` to the room; losers get an error ack. Use server timestamps for the deadline; consider an anti-sniping extension. Persist every bid for audit.

Q7. How do you handle reconnects and missed events?

Socket.IO reconnects automatically; on `connect` the client re-joins rooms and requests a state snapshot (current highest bid, end time, recent bids), or the server tracks a last-event ID. Don't assume delivery of every event.

Q8. How do you stop a flood of events?

Rate limit per socket/user, validate payload size, throttle broadcasts (batch updates every N ms), disconnect abusive clients, and cap rooms/connections per user.

Edge cases and gotchas

- Registering `socket.on` inside React effects without cleanup causes duplicate handlers and duplicated UI updates.
- CORS must be configured on the Socket.IO server separately from Express.

- Without sticky sessions, long-polling connections break behind multiple instances.
- Connection counts consume file descriptors and memory; tune OS limits.
- `io.emit` broadcasts to everyone; `socket.broadcast.emit` excludes the sender; `socket.emit` replies to one.
- Browsers throttle background tabs' timers; do not rely on client timers for auction end.
- Reverse proxies need WebSocket upgrade headers and long idle timeouts (and Cloud Run/ALB have maximum durations, so clients must reconnect).

Rapid-fire round

Question	Answer
Default Socket.IO transports	HTTP long-polling first, then upgrade to WebSocket
What is a heartbeat	Ping/pong to detect dead connections
What is an acknowledgement	A callback confirming receipt/processing of an event
WebSocket URL schemes	`ws://` and `wss://` (TLS)
Why use wss	Encryption, and proxies/firewalls pass it more reliably

MongoDB

> Link to your resume: Alethea (MongoDB). Be ready to justify why a document store for logs and incident data.

Q1. SQL vs NoSQL, and when would you choose MongoDB?

MongoDB stores flexible JSON-like documents (BSON), scales horizontally with sharding, and suits evolving schemas, nested data and high write volumes (logs, events, content). SQL suits strongly relational data, complex joins and multi-row ACID guarantees. MongoDB does support multi-document transactions, but they cost performance, so model data to avoid needing them.

Q2. Embedding vs referencing.

Embed when data is read together, has a 1-to-few relationship and similar lifecycle (address in user). Reference when data is large, unbounded, shared or updated independently (orders to users). Anti-pattern: unbounded arrays that grow forever, because documents are limited to **16 MB** and large arrays hurt performance.

Q3. How do indexes work and what is the ESR rule?

MongoDB uses B-tree indexes. Types: single field, compound, multikey (arrays), text, geospatial (2dsphere), hashed, TTL, partial, unique. For compound indexes follow **ESR**: **E**quality fields first, then **S**ort fields, then **R**ange fields. Use `explain("executionStats")` and look for `IXSCAN` rather than `COLLSCAN`, and compare `totalDocsExamined` with `nReturned`.

Q4. Explain the aggregation pipeline.

A sequence of stages that transform documents: `$match`, `$group`, `$project`, `$sort`, `$limit`, `$unwind`, `$lookup` (left outer join), `$facet`, `$addFields`. Put `$match` and `$limit` early so indexes are used and fewer documents flow through the pipeline.

```
db.logs.aggregate([
  { $match: { level: "ERROR", ts: { $gte: ISODate("2026-01-01") } } },
  { $group: { _id: "$service", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 5 }
])
```

Q5. Replica sets, sharding and consistency.

A replica set has one primary and secondaries with automatic failover via elections; reads go to the primary by default. **Write concern** (`w: "majority"`) and **read concern** control durability and consistency. Sharding splits data across shards by a shard key (needs high cardinality, even distribution, query isolation; a monotonically increasing key creates a hot shard).

Q6. Mongoose: schema, populate, lean.

Mongoose adds schemas, validation, middleware and models over the driver. `populate` performs application-level joins (extra queries, not a DB join). `.lean()` returns plain objects and is faster for read-only use.

Q7. Pagination approaches.

`skip/limit` gets slower as skip grows (the server still scans skipped docs). Prefer range pagination on an indexed field (`_id > lastId`, sorted).

Q8. What is a TTL index? Where would you use it for logs?

An index with `expireAfterSeconds` on a date field; a background task deletes expired documents (not instant, runs about every 60 seconds). Good for log retention, sessions, OTPs.

Q9. How do you model high-volume logs/time-series?

Use time-series collections (MongoDB 5.0+) or bucket pattern, index on time and service, TTL for retention, and avoid unbounded embedded arrays. Capped collections are fixed-size and FIFO.

Edge cases and gotchas

- `_id` is auto-indexed and unique; `ObjectId` embeds a timestamp, roughly sortable by creation.
- Index order in a compound index matters; an index on `{a, b}` supports queries on `a` and `{a, b}` but not `b` alone.
- Too many indexes slow writes and use RAM.
- `$lookup` on unindexed foreign fields is slow; large `$group` may exceed 100 MB stage memory without `allowDiskUse`.
- Operator injection: `{ "password": { "$ne": null } }` in a login body; sanitise input and enforce types.
- No schema does not mean no schema design; schema validation (`$jsonSchema`) exists.
- Updates are atomic at the single-document level.

Rapid-fire round

Question	Answer
Default port	27017
BSON vs JSON	Binary JSON with extra types (Date, ObjectId, Decimal128)
`find` vs `findOne`	Cursor of many vs first match
`$set` vs replace	Update fields vs replace the whole document
Covered query	Answered entirely from an index
CAP behaviour	CP by default (consistency over availability during partition)

Redis

> Link to your resume: BehaveGuard (Redis for session management and API rate limiting) and Alethea (Redis). Rate limiting and sessions are almost guaranteed follow-ups.

Q1. What is Redis and why is it fast?

An in-memory data-structure store used as cache, session store, message broker, and primitive database. Fast because data lives in RAM, operations are simple, it uses an efficient single-threaded event loop for command execution (no lock contention), and I/O multiplexing. Newer versions use extra threads for network I/O.

Q2. Data structures and use cases.

Strings (cache, counters), Hashes (objects, sessions), Lists (queues), Sets (unique members, tags), Sorted Sets (leaderboards, delayed jobs, sliding-window rate limits), Bitmaps/HyperLogLog (flags, approximate unique counts), Streams (durable event log, consumer groups), Geo, Pub/Sub.

Q3. Persistence: RDB vs AOF.

RDB takes point-in-time snapshots (compact, fast restart, may lose data since the last snapshot). AOF appends every write (more durable with `appendfsync everysec`, larger, slower restart). Using both is common. If it is purely a cache, persistence may be disabled.

Q4. Eviction policies and TTL.

When `maxmemory` is reached Redis evicts according to policy: `noeviction`, `allkeys-lru`, `volatile-lru`, `allkeys-lfu`, `volatile-ttl`, `allkeys-random`. Keys expire via lazy deletion on access plus periodic sampling of expiring keys.

Q5. Caching strategies and failure modes.

Cache-aside (app reads cache, falls back to DB, then populates), write-through, write-behind. Problems: **cache stampede** (many requests rebuild an expired hot key; fix with locking/single-flight, jittered TTLs, early refresh), **cache penetration** (queries for nonexistent keys; cache nulls or use Bloom filters), **cache avalanche** (many keys expire together; add TTL jitter), stale data (invalidate on write).

Q6. How do you implement session management with Redis? (resume)

On login, generate a random session ID (cryptographically secure), store a hash `session:<id>` with user ID and metadata plus a TTL (e.g. sliding expiration by refreshing TTL on activity), and send the ID in an `HttpOnly`, `Secure`, `SameSite` cookie. Because the app servers are stateless (e.g. on Cloud Run), Redis acts as the shared store; logout deletes the key; you can list/kill all sessions of a user using a per-user set.

Q7. How do you implement rate limiting with Redis? (resume)

Fixed window: `INCR key` then `EXPIRE` on first hit; simple but allows bursts at window edges. Sliding window log: a sorted set of timestamps (`ZADD`, `ZREMRANGEBYSCORE`, `ZCARD`); precise but more memory. Sliding window counter: weighted combination of the current and previous window. Token bucket: tokens refill at a steady rate, allows bursts; usually implemented in a **Lua script** so check-and-update is atomic. Key by user ID / API key / IP, return `429` with `Retry-After`.

```
-- fixed window, atomic
local c = redis.call("INCR", KEYS[1])
if c == 1 then redis.call("EXPIRE", KEYS[1], ARGV[1]) end
return c
```

Q8. Transactions and atomicity.

`MULTI/EXEC` queues commands and runs them without interleaving (no rollback on logic errors); `WATCH` gives optimistic locking; Lua scripts (`EVAL`) run atomically and are the best tool for check-then-set logic.

Q9. Distributed locks.

`SET lock:key <unique-token> NX PX 30000` to acquire; release only if the token matches (Lua compare-and-delete) so you never delete someone else's lock. Locks need TTLs for crash safety, and a long-running holder can lose the lock mid-work (use fencing tokens). Redlock across multiple nodes is debated; for strict correctness prefer database constraints.

Q10. Pub/Sub vs Streams.

Pub/Sub is fire-and-forget: subscribers that are offline miss messages. Streams persist messages with IDs and consumer groups, supporting acknowledgements and replay.

Q11. Replication, Sentinel and Cluster.

Replication: primary with asynchronous replicas. Sentinel: monitoring and automatic failover for a primary-replica setup. Cluster: shards data over 16384 hash slots across nodes for horizontal scaling. Managed options: ElastiCache, Memorystore.

Edge cases and gotchas

- `KEYS *` blocks the server on big datasets; use `SCAN`.
- `INCR` followed by a separate `EXPIRE` can leave a key with no TTL if the process crashes between them; use a Lua script or `SET key 1 EX 60 NX`.
- Big keys (huge lists or hashes) and `HGETALL` on them hurt latency.
- Cache and DB can drift: decide an invalidation strategy up front.
- Redis is single-threaded for commands: one slow command (`SORT`, large `LRANGE`) blocks everyone.
- Don't rely on Redis as the sole source of truth without persistence and replication.
- Multi-key operations in Cluster must hash to the same slot (hash tags `{user1}`).

Rapid-fire round

Question	Answer
Default port	6379
Redis vs Memcached	Redis has rich types, persistence, replication; Memcached is simple, multi-threaded cache
`EXPIRE` vs `TTL`	Set expiry vs read remaining time
What is a hot key	A key receiving disproportionate traffic
Why not use Redis for everything	RAM is costly and data size is bounded by memory
Lazy vs active expiry	On access vs background sampling

PostgreSQL and SQL

> Link to your resume: BehaveGuard (PostgreSQL for enrollment/verification data). Expect SQL writing, indexing and transaction questions.

Q1. What are ACID properties?

Atomicity (all or nothing), **Consistency** (constraints always hold), **Isolation** (concurrent transactions don't see each other's partial work), **Durability** (committed data survives crashes via the write-ahead log).

Q2. Transaction isolation levels and anomalies.

Read Committed (Postgres default): each statement sees only committed data (non-repeatable reads possible). Repeatable Read: snapshot per transaction (in Postgres it also prevents phantom reads but can raise serialisation failures). Serializable: behaves as if transactions ran one after another (uses SSI; retry on failure). Anomalies: dirty read (Postgres never allows), non-repeatable read, phantom read, write skew.

Q3. What is MVCC?

Multi-Version Concurrency Control: updates create new row versions instead of overwriting, so readers don't block writers and writers don't block readers. Old versions (dead tuples) are cleaned by **VACUUM**/autovacuum; neglecting it causes table bloat.

Q4. Index types and when to use them.

B-tree (default: equality, range, sorting), Hash (equality only), GIN (arrays, JSONB, full-text), GiST (geometric, ranges), BRIN (huge naturally ordered tables like time-series). Also composite, partial (`WHERE active`), covering (`INCLUDE`), expression indexes. Index trade-off: faster reads, slower writes and more storage. Column order in a composite index matters (leftmost prefix).

Q5. How do you debug a slow query?

`EXPLAIN (ANALYZE, BUFFERS)`: look for sequential scans on big tables, wrong row estimates (run `ANALYZE`), nested loops over huge sets, sorts spilling to disk, missing indexes, N+1 queries from the app, `SELECT *`, functions on indexed columns (`WHERE lower(email)=...` needs an expression index), `OFFSET` pagination.

Q6. JOIN types and WHERE vs HAVING.

INNER (matching rows), LEFT/RIGHT (all from one side plus matches), FULL OUTER, CROSS, SELF join. `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation.

Q7. DELETE vs TRUNCATE vs DROP; UNION vs UNION ALL.

DELETE removes rows (can filter, logged per row, triggers fire); TRUNCATE empties a table quickly (no filter); DROP removes the table itself. `UNION` removes duplicates (extra sort/hash); `UNION ALL` keeps them and is faster.

Q8. Connection pooling: why and how?

Each Postgres connection is a process with real memory cost, so thousands of direct connections hurt. Use an app-level pool (`pg.Pool`) and/or PgBouncer. In serverless/Cloud Run where instances scale quickly, cap pool size per instance so (instances x pool) stays under `max_connections`.

Q9. JSONB vs separate columns.

JSONB is binary, indexable (GIN), good for semi-structured attributes. Use relational columns for data you filter, join or constrain heavily.

Q10. How would you design tables for biometric enrolment/verification? (resume)

`users(id, created_at, ...)`, `enrollments(id, user_id, feature_vector/model_ref, version, created_at)`, `verification_attempts(id, user_id, score, decision, threshold, ts, ip/device)` with indexes on `(user_id, ts)`. Store derived features or model references rather than raw keystroke text; apply retention policy and access control because behavioural biometrics are sensitive personal data.

Q11. Write these SQL queries.

```
-- 2nd highest salary
SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);

-- Top 3 per department
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) r
  FROM emp) t WHERE r <= 3;

-- Delete duplicate emails, keep lowest id
DELETE FROM users a USING users b
WHERE a.email = b.email AND a.id > b.id;

-- Upsert
INSERT INTO counters(k, v) VALUES ('x', 1)
ON CONFLICT (k) DO UPDATE SET v = counters.v + 1;
```

Edge cases and gotchas

- `NULL = NULL` is not true; use `IS NULL`. `NOT IN (subquery)` returns nothing if the subquery contains a NULL; use `NOT EXISTS`.
- `COUNT(*)` counts rows; `COUNT(col)` ignores NULLs.
- Postgres has no clustered primary index; `CLUSTER` is a one-time reorder.
- Long-running transactions block VACUUM and bloat tables.
- Deadlocks: two transactions lock rows in opposite order; fix by consistent lock order; Postgres aborts one.
- `SERIAL` vs `IDENTITY`; UUID primary keys randomise B-tree inserts.
- Foreign keys are not auto-indexed in Postgres; index them yourself.
- `LIMIT` without `ORDER BY` is non-deterministic.

Rapid-fire round

Question	Answer
Default port	5432
Primary key vs unique	PK is unique + not null, one per table
Normalisation (1NF-3NF) in one line	Atomic values; no partial dependency; no transitive dependency
View vs materialised view	Stored query vs stored result needing refresh
What is a CTE	`WITH` named subquery for readability/recursion
`ROW_NUMBER` vs `RANK` vs `DENSE_RANK`	Unique sequence; gaps after ties; no gaps

Git

> Link to your resume: 14+ patches merged into Git itself through the mailing-list process (v2.54.0), plus daily use in every project. Interviewers will test fundamentals hard here because you claim Git contributions.

Q1. How does Git store data internally?

Git is a content-addressable object database. Four object types: **blob** (file contents), **tree** (directory listing of blobs and trees), **commit** (pointer to a tree, parent commit(s), author, message) and **tag**. Objects are named by SHA hash of their content (SHA-1 by default; SHA-256 is supported). Branches and tags are just refs (small files containing a commit hash); `HEAD` points to the current branch or commit. Git stores snapshots, not diffs (packfiles use delta compression for storage).

Q2. Merge vs rebase. When do you use each?

Merge combines histories with a merge commit (or fast-forward), preserving the true history. Rebase replays your commits on top of another base, producing a linear history but **rewriting commits** (new hashes). Golden rule: never rebase commits that others already pulled/shared. Rebase to tidy a local feature branch before review; merge (or squash) into shared branches.

Q3. reset (soft/mixed/hard) vs revert vs checkout/restore.

`reset --soft` moves the branch pointer, keeps index and working tree. `--mixed` (default) also resets the index. `--hard` also discards working-tree changes (dangerous). `revert` creates a new commit undoing an earlier one, safe for shared history. `restore`/`checkout -- file` discard changes to files.

Q4. fetch vs pull. What is a fast-forward?

`fetch` downloads remote changes without touching your branch. `pull` = fetch + merge (or rebase with `--rebase`). A fast-forward merge just moves the branch pointer when the target is a direct descendant, with no merge commit.

Q5. How do you undo a bad commit, a lost commit, or a bad force-push?

Local unpushed: `git commit --amend` or `reset`. Pushed and shared: `git revert`. Lost commits after a hard reset or deleted branch: `git reflog` lists previous HEAD positions; `git branch recover <hash>`. Prefer `git push --force-with-lease` over `--force` because it refuses to overwrite commits you haven't seen.

Q6. cherry-pick, stash, bisect, interactive rebase.

`cherry-pick <hash>` applies one commit elsewhere. `stash` shelves uncommitted work. `bisect` binary-searches history for the commit that introduced a bug (`git bisect start/good/bad`, or `run <script>`). `rebase -i` lets you reorder, squash, fixup, edit, reword commits.

Q7. How do you resolve merge conflicts?

Open conflicted files (markers `<<<<<<<`, `=======`, `>>>>>>>`), choose or combine changes, `git add`, then `git commit` (or `git rebase --continue`). Use `git mergetool`, and `git merge --abort` to back out. `git rerere` records resolutions to reuse.

Q8. Explain the mailing-list contribution workflow you used for Git. (resume)

Git development happens on the `git@vger.kernel.org` mailing list rather than through pull requests. Typical flow: clone git.git, create a topic branch off `master` (or the relevant integration branch), make small focused commits with imperative-mood messages and `Signed-off-by:` (Developer Certificate of Origin), run the test suite (`make test` or specific `t/tXXXX-*.sh`), follow `Documentation/CodingGuidelines` and `SubmittingPatches`, generate patches with `git format-patch` (with a cover letter via `--cover-letter`), send with `git send-email` (or use **GitGitGadget** to turn a GitHub PR into list mail), reply to reviewer comments inline, and send revisions as `-v2`, `-v3` with a changelog (`git range-diff` shows what changed between versions). The maintainer picks accepted topics into `seen`, then `next`, then `master`, and posts "What's cooking in git.git" updates; releases follow.

Be ready with: the titles of 2-3 patches, what each fixed, what review feedback you received and how you responded, which areas of the code you touched, and what you learned about review culture.

Q9. Branching strategies.

Git Flow (develop/release/hotfix branches, heavier), GitHub Flow (short feature branches off main with PRs), trunk-based development (small commits to main behind feature flags; best fit for CI/CD).

Q10. .gitignore, tags, submodules, hooks.

`.gitignore` only affects untracked files (use `git rm --cached` for already tracked ones). Annotated tags mark releases (`git tag -a v1.0 -m ""`). Submodules embed another repo at a pinned commit. Hooks (`pre-commit`, `commit-msg`, `pre-push`) run scripts at events; Husky/lint-staged are common wrappers.

Edge cases and gotchas

- Detached HEAD: you checked out a commit/tag; make a branch before committing or work is easily lost.
- Committed a secret: deleting it in a new commit isn't enough; rotate it and rewrite history (`git filter-repo`/BFG).
- `git add -p` stages hunks interactively for clean commits.
- Empty directories are not tracked.
- Case-insensitive file systems cause rename problems.
- Line endings: `core.autocrlf`, `.gitattributes`.
- `git pull` with diverged branches requires choosing merge or rebase.
- `git log --oneline --graph --all`; `git blame -L` ; `git show`; `git diff --staged`.

Rapid-fire round

Question	Answer
Git vs GitHub	Version control tool vs hosting platform
Staging area (index)	Prepared snapshot for the next commit
`git clone --depth 1`	Shallow clone, only latest history
`git commit --amend`	Edit the last commit
`HEAD~2` vs `HEAD^2`	Two commits back vs second parent of a merge
Squash merge	Collapses a branch into one commit on the target

Docker

> Link to your resume: Docker deployment pipeline optimisation at Procurpal (10 min to 3.6 min, 5x smaller artifacts), BehaveGuard on Cloud Run, Alethea.

Q1. Image vs container; Docker vs VM.

An image is a read-only layered template; a container is a running (isolated) instance of an image with a thin writable layer. Containers share the host kernel and isolate with namespaces and cgroups, so they are lightweight and start in milliseconds; VMs run a full guest OS on a hypervisor, with stronger isolation but more overhead.

Q2. How do image layers and build cache work?

Each Dockerfile instruction creates a layer; unchanged layers are reused from cache until an instruction (or the files it copies) changes, after which all following layers rebuild. So order matters: copy dependency manifests and install first, copy source last.

```
FROM node:22-slim
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
CMD ["node", "server.js"]
```

Q3. How did you cut deployment from 10 min to 3.6 min and artifacts by 5x? (resume)

Explain only what you actually did, using levers like these: multi-stage builds (build with dev tools, ship only runtime), slim/alpine/distroless base images, `npm ci --omit=dev` / pruning dev dependencies, bundling and tree-shaking (esbuild), `.dockerignore`, layer ordering for cache hits, caching dependencies/layers in CI, parallel steps, smaller Lambda packages (shared layers), removing unused assets. Prepare the before/after numbers and *how you measured* them (CI timings, image size from `docker images`).

Q4. Multi-stage builds.

Several `FROM` stages in one Dockerfile; copy only artefacts from the build stage into the final minimal stage with `COPY --from=build`. Gives smaller, safer images with no compilers or source code.

Q5. CMD vs ENTRYPOINT; COPY vs ADD; ARG vs ENV.

`ENTRYPOINT` is the fixed executable; `CMD` supplies default arguments (overridable at `docker run`). `COPY` copies files; `ADD` also extracts tarballs and fetches URLs (prefer COPY). `ARG` exists at build time only; `ENV` persists in the image and container. Use exec form (`["node","app.js"]`) so the app is PID 1 and receives signals.

Q6. Volumes vs bind mounts; networking.

Volumes are managed by Docker (persistent data, portable). Bind mounts map a host path (dev hot-reload). tmpfs is in memory. Networks: bridge (default, container-to-container by name on user-defined bridges), host, none, overlay (multi-host). Publish ports with `-p host:container`.

Q7. Docker Compose?

Defines multi-container apps (app + Postgres + Redis) in YAML with services, networks, volumes, env, and `depends_on` (with healthchecks for readiness). Used for local development and simple deployments.

Q8. How do you keep images secure and small?

Minimal base image, multi-stage, non-root `USER`, pinned versions (digest), no secrets baked in (use runtime env/secret managers), scan with Trivy/Docker Scout, `.dockerignore`, read-only filesystem where possible, update base images regularly.

Q9. Docker with AWS Lambda?

Lambda supports container images (up to 10 GB) from ECR using AWS base images that include the runtime interface client; versus zip packages (50 MB zipped, 250 MB unzipped including layers). Containers help with big dependencies and consistent tooling.

Edge cases and gotchas

- `latest` tag is mutable and unpredictable; pin versions.
- Secrets in `ENV`, `ARG` or a `COPY`'d `.env` stay in image layers forever (`docker history`).
- A container exits when PID 1 exits; shell-form `CMD` wraps in `sh -c` and may swallow SIGTERM.
- Bind your server to `0.0.0.0`, not `127.0.0.1`, or it is unreachable from outside the container (a frequent Cloud Run failure).
- Alpine uses musl libc, causing issues with some native modules; slim (Debian) is often safer.
- Data in the container's writable layer disappears with the container; use volumes.
- Large build context (no `.dockerignore`) slows every build.
- Apple Silicon builds `arm64` images; target `linux/amd64` for most cloud runtimes (`--platform`).

Rapid-fire round

Question	Answer
`docker stop` vs `docker kill`	SIGTERM with grace period vs SIGKILL
`docker exec`	Run a command inside a running container
Registry examples	Docker Hub, ECR, Artifact Registry, GHCR
What is a dangling image	Untagged leftover layer set; prune with `docker image prune`
HEALTHCHECK	Command Docker runs to mark a container healthy/unhealthy
Containers vs images count	Many containers can run from one image

CI/CD and GitHub Actions

> Link to your resume: BehaveGuard (Docker with GitHub Actions CI/CD to Google Cloud Run), Procurpal deployment pipeline.

Q1. CI vs Continuous Delivery vs Continuous Deployment.

CI: every change is automatically built and tested, merged often. Continuous Delivery: every passing change is releasable, deployed with a manual approval. Continuous Deployment: every passing change goes to production automatically.

Q2. Describe your pipeline end to end. (resume)

A sensible answer: on push/PR trigger, checkout, install with cache, lint and type-check, unit tests, build Docker image, tag with the commit SHA, push to a registry (Artifact Registry), deploy to Cloud Run (`gcloud run deploy`) on merge to main, run a smoke test/health check, notify on failure, and keep rollback to the previous revision. Describe what *you* configured.

Q3. Explain GitHub Actions concepts.

Workflow (YAML in `.github/workflows/`), **events** (`push`, `pull_request`, `workflow_dispatch`, `schedule`), **jobs** (run on a runner, parallel by default, `needs:` for order), **steps** (shell commands or reusable **actions**), **runners** (GitHub-hosted or self-hosted), **matrix** builds, **secrets** and variables, **artifacts** and **cache**, **environments** with protection rules and approvals, `concurrency` groups to cancel superseded runs.

```
name: deploy
on:
  push: { branches: [main] }
permissions: { contents: read, id-token: write }
jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci && npm test
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
          service_account: ${{ secrets.DEPLOY_SA }}
      - uses: google-github-actions/deploy-cloudrun@v2
        with: { service: api, image: REGION-docker.pkg.dev/PROJ/repo/api:${{ github.sha }} }
```

Q4. How do you handle secrets and cloud credentials securely?

Store secrets in GitHub encrypted secrets/environments (never in the repo); prefer **OIDC federation** (Workload Identity Federation for GCP, IAM role with web identity for AWS) over long-lived JSON keys or access keys; least-privilege service accounts; mask logs; restrict who can edit workflows; pin third-party actions to a commit SHA; be careful with `pull_request_target` and forks (a classic secret-leak vector).

Q5. Deployment strategies.

Rolling (replace gradually), blue-green (two environments, switch traffic, instant rollback), canary (small percentage first, observe, then increase). Cloud Run supports traffic splitting between revisions for canary and instant rollback.

Q6. How do you speed up pipelines?

Dependency and Docker layer caching, parallel jobs, running only affected tests, matrix sharding, smaller images, skipping on docs-only changes with path filters, reusing build artifacts between jobs.

Q7. What makes a pipeline reliable?

Deterministic builds (lockfiles, pinned versions), fast feedback, flaky-test management, immutable image tags by commit SHA, environment parity, automated rollback and health checks, notifications, and database migrations that are backward-compatible.

Edge cases and gotchas

- Cache keys: include lockfile hash so the cache invalidates properly.
- Secrets are not exposed to workflows from forked PRs by default.
- Jobs run on fresh runners; share files via artifacts or caches.
- `needs` creates dependency; without it jobs run in parallel.
- Migrations: expand/contract pattern so old and new code both work during rollout.
- Never deploy `latest`; deploy a SHA so rollbacks are exact.

Rapid-fire round

Question	Answer
Runner	Machine executing a job
`workflow_dispatch`	Manual trigger
Artifact vs cache	Output to keep/share vs dependency data to reuse
Rollback on Cloud Run	Route traffic back to a previous revision
Infrastructure as Code examples	Terraform, CloudFormation, Pulumi

AWS (Lambda, DynamoDB, S3, EC2)

> Link to your resume: Procurpal used Lambda microservices and DynamoDB; your skills list includes S3 and EC2. Know serverless trade-offs and DynamoDB data modelling cold.

Q1. How does AWS Lambda work? Key limits and cold starts.

You upload code; Lambda runs it on demand in managed micro-VM execution environments, scaling per concurrent request, and billing per request and per GB-second. A **cold start** is the setup of a new environment (download code, start runtime, run init code); warm environments are reused. Limits to remember: max **15 minutes** timeout, memory 128 MB to 10 GB (CPU scales with memory), **6 MB** synchronous payload, zip package 50 MB (250 MB unzipped), container image up to 10 GB, `/tmp` 512 MB default (configurable to 10 GB). Reduce cold starts with smaller packages, lighter runtimes, init outside the handler, provisioned concurrency, avoiding VPC where unnecessary, and SnapStart (Java and some other runtimes).

Q2. How do you design reliable Lambda functions?

Idempotent handlers (events are delivered **at least once**), explicit timeouts and retries, dead-letter queues/on-failure destinations, reserved concurrency to protect downstreams, structured logging to CloudWatch, least-privilege execution roles, environment variables/Secrets Manager for config, and reuse SDK clients across invocations by creating them outside the handler.

Q3. Lambda triggers and common architecture.

API Gateway/Function URLs (HTTP), S3 events, DynamoDB Streams, SQS, SNS, EventBridge (events and schedules), Kinesis. Typical: API Gateway to Lambda to DynamoDB, with SQS for decoupling and EventBridge Scheduler for cron.

Q4. DynamoDB fundamentals: keys, indexes and capacity.

A key-value/document NoSQL database with single-digit millisecond latency at scale. Primary key = **partition key** (hashed to choose the partition) optionally plus **sort key** (orders items within a partition). **GSI**: alternate partition/sort keys, eventually consistent, own capacity; **LSI**: same partition key with alternate sort key, created at table creation. Capacity: on-demand (pay per request) or provisioned (RCU/WCU, with auto scaling). Item size limit **400 KB**.

Q5. Query vs Scan; consistency; transactions.

`Query` reads items for a partition key (optionally with sort-key conditions), efficient. `Scan` reads the entire table and is expensive; avoid in production paths. Reads are eventually consistent by default; **strongly consistent reads** are available on base table and LSI (not GSI). `TransactWriteItems` gives ACID across items (up to 100). `ConditionExpression` gives optimistic locking/uniqueness (`attribute_not_exists(pk)`).

Q6. How do you choose a partition key? What is a hot partition?

Pick a high-cardinality key with evenly distributed access. A hot partition arises when a few keys get most traffic (e.g. a date or status as the key). Mitigations: write sharding (suffix), better key design, caching (DAX), on-demand capacity.

Q7. DynamoDB design for your e-signing sync. (resume)

Possible design: partition key = `documentId`, attributes: provider request ID, status, last synced time, version. A cron/Lambda queries documents in non-terminal states (via a GSI on `status` + `updatedAt`) and syncs them with Qsign, using conditional updates to avoid overwriting newer state. Adapt this to what you really built.

Q8. SQL vs DynamoDB: when do you use which?

DynamoDB when access patterns are known and predictable, you need massive scale and low latency, minimal ops. SQL when you need ad hoc queries, joins, complex reporting, flexible access patterns. DynamoDB requires designing tables around queries (often single-table design) and has limited secondary indexes.

Q9. S3 essentials.

Object storage in buckets (11 nines durability, strong read-after-write consistency). Storage classes (Standard, IA, Glacier tiers, Intelligent-Tiering) with lifecycle rules; versioning; encryption (SSE-S3/KMS); **presigned URLs** for temporary upload/download without exposing credentials; event notifications to Lambda/SQS; bucket policies vs IAM policies; Block Public Access; CORS for browser uploads; multipart upload for large files.

Q10. EC2 essentials.

Virtual machines: choose instance family/size, an AMI, storage (EBS volumes; instance store is ephemeral), **security groups** (stateful, instance-level allow rules) versus NACLs (stateless, subnet-level), key pairs/SSM Session Manager for access, Elastic IP, Auto Scaling Groups with load balancers (ALB/NLB), pricing options (on-demand, reserved/savings plans, spot).

Q11. IAM basics.

Users, groups, **roles** (assumed temporarily, used by services like Lambda/EC2), and policies (JSON: Effect, Action, Resource, Condition). Principle of least privilege; explicit deny wins; no long-lived access keys when roles work; MFA for humans.

Q12. Serverless vs containers vs VMs: how do you choose?

Serverless (Lambda): spiky/event-driven, least ops, but limits and cold starts. Containers (ECS/Fargate, Cloud Run): long-running or custom runtime, consistent packaging. VMs (EC2): full control, steady heavy workloads, special software.

Edge cases and gotchas

- Lambda retries: async invocations retry twice by default; SQS-triggered failures reappear after the visibility timeout; design for duplicates.
- Lambda in a VPC needs NAT/VPC endpoints to reach the internet or AWS services.
- DynamoDB `Scan` + `Filter` still reads (and charges for) all items before filtering.
- A GSI throttling can throttle base-table writes.
- DynamoDB reserved words need `ExpressionAttributeNames`.
- DynamoDB TTL deletes are eventual (typically within days after expiry).
- S3 bucket names are globally unique; presigned URLs inherit the signer's permissions and expire.
- Lambda concurrency limit per account/region can throttle other functions.
- Time-limited EC2 key pair loss: use SSM or EBS recovery.

Rapid-fire round

Question	Answer
Lambda pricing basis	Requests plus duration (GB-seconds)
DynamoDB consistency default	Eventually consistent reads
S3 object size max	5 TB (5 GB per single PUT; multipart beyond)
Security group stateful or stateless	Stateful
CloudWatch	Logs, metrics, alarms
SQS standard vs FIFO	At-least-once, best-effort order vs exactly-once processing, strict order
SNS vs SQS	Pub/sub fan-out vs queue
Region vs AZ	Geographic area vs isolated data centre(s) within it

Google Cloud Run

> Link to your resume: BehaveGuard was containerised and deployed on Cloud Run with GitHub Actions.

Q1. What is Cloud Run and how is it different from Lambda, GKE and App Engine?

A managed serverless platform that runs **stateless containers**, scales automatically (including to zero), and bills per request/CPU/memory time. Unlike Lambda, you bring any container and each instance handles many concurrent requests. GKE is full Kubernetes (more control and ops), App Engine is a PaaS for source-based apps, Cloud Functions runs single functions (now built on Cloud Run).

Q2. How does scaling and concurrency work?

Each instance can serve multiple requests concurrently (default 80, configurable up to 1000). Cloud Run adds instances based on concurrency and CPU use, between **min instances** (set above 0 to avoid cold starts) and **max instances** (protect downstreams like the DB). Cold start = container start plus app boot; reduce with smaller images, lazy initialisation, startup CPU boost.

Q3. What are the container requirements?

Listen on `0.0.0.0` on the port given by the `PORT` environment variable, be stateless (local disk is ephemeral and in-memory), start quickly, and handle SIGTERM for graceful shutdown. Anything that must persist goes to Cloud SQL/Redis/Cloud Storage.

Q4. Services vs Jobs; revisions and traffic splitting.

Services respond to HTTP requests; Jobs run containers to completion (batch, migrations, cron via Cloud Scheduler). Every deploy creates an immutable **revision**; you can split traffic (canary), pin tags, and roll back instantly.

Q5. Configuration and security.

Environment variables, secrets from **Secret Manager** (mounted or env), runtime **service account** with least privilege, **IAM invoker** role to make a service private, ingress settings, VPC connector/Direct VPC egress to reach private resources (Cloud SQL, Memorystore Redis). Images live in Artifact Registry.

Q6. Connecting to PostgreSQL and Redis from Cloud Run. (resume)

Cloud SQL via the built-in Cloud SQL connection (Unix socket) or private IP with VPC access; Memorystore for Redis requires VPC connectivity. Because instances scale out, limit DB pool size per instance and use `max-instances`, since many instances times pool size can exhaust `max_connections`. State such as sessions must be in Redis, not in instance memory.

Q7. Can you run WebSockets on Cloud Run?

Yes, WebSockets are supported, but connections are subject to the request timeout (configurable, up to 60 minutes), so clients must reconnect, and multiple instances need a shared backplane (Redis) for broadcasts.

Q8. How does CPU allocation/billing work?

By default CPU is allocated only during request processing (request-based billing); background work after the response may be throttled. "Instance-based" (always-allocated CPU) suits background tasks and WebSockets but costs more.

Edge cases and gotchas

- Container fails to start if it doesn't listen on `$PORT` within the startup timeout.
- Writing to the filesystem uses memory (in-memory filesystem).
- Background threads after sending the response may be paused with request-based billing.
- Scaling to zero causes cold starts on the first request.
- Maximum request timeout is configurable (default 5 minutes).

► Each revision is immutable; env var changes create a new revision.

Rapid-fire round

Question	Answer
Cloud Run default concurrency	80
Where do images go	Artifact Registry
Scale to zero	Min instances 0
Cron on Cloud Run	Cloud Scheduler triggers a service or Job
Private service	Require authentication / IAM invoker

Python and scikit-learn

> Link to your resume: Python is listed under Languages; BehaveGuard (Scikit-learn, feature engineering), MyAiTree (Python AI backend). Section 2 covers the ML-metrics defence for your project.

Q1. List vs tuple vs set vs dict. Mutable vs immutable.

List: ordered, mutable. Tuple: ordered, immutable, hashable if elements are. Set: unordered unique elements, O(1) membership. Dict: key-value, insertion ordered (3.7+), O(1) average lookups. Immutable: int, str, tuple, frozenset; mutable: list, dict, set.

Q2. What is the GIL?

The Global Interpreter Lock lets only one thread execute Python bytecode at a time in CPython. Threads help I/O-bound work, not CPU-bound work; use `multiprocessing`, native extensions (NumPy), or async I/O. (Free-threaded builds exist in recent Python versions but are not the default everywhere.)

Q3. Decorators, generators, context managers.

Decorator: a function that wraps another to add behaviour (`@app.route`, `@lru_cache`). Generator: a function using `yield` that produces values lazily (memory-efficient). Context manager: `with` statement resource handling via `__enter__/__exit__` or `contextlib.contextmanager`.

Q4. `is` vs `==`; shallow vs deep copy; mutable default argument bug.

`==` compares values, `is` compares identity. `copy.copy` is shallow, `copy.deepcopy` is deep. Bug: `def f(x, items=[])` shares one list across calls; use `None` and create inside.

Q5. *args and **kwargs; list comprehension vs generator expression.

`*args` collects extra positional arguments into a tuple; `**kwargs` collects keyword arguments into a dict. `[x*x for x in data]` builds a list; `(x*x for x in data)` yields lazily.

Q6. Virtual environments and dependency management.

`venv`/`virtualenv`, `pip`, `requirements.txt` or `pyproject.toml` (poetry/uv) isolate dependencies per project; pin versions for reproducible Docker builds.

Q7. scikit-learn workflow and pitfalls.

Split data (train/validation/test), build a `Pipeline` (scaler + model) so preprocessing is fit only on training data, tune hyperparameters with cross-validation (`GridSearchCV`), evaluate with the right metric, save with `joblib`. **Data leakage** (fitting a scaler on all data, features derived from the target, same user in train and test) inflates results. For multi-user behavioural data, split **by user/session**, not by random rows.

Q8. Overfitting vs underfitting; bias-variance.

Overfitting: great on training data, poor on unseen data (high variance); fix with more data, regularisation, simpler models, cross-validation, early stopping. Underfitting: poor on both (high bias); fix with richer features or models.

Q9. Precision, recall, F1, ROC/AUC, balanced accuracy.

Precision = TP/(TP+FP); recall = TP/(TP+FN); F1 = harmonic mean. Balanced accuracy = average of recall on each class (robust to class imbalance). ROC plots TPR vs FPR across thresholds; AUC is the area (threshold-independent ranking quality). For authentication also use **FAR** (false accept rate), **FRR** (false reject rate) and **EER** (equal error rate, where FAR = FRR).

Q10. Class imbalance handling.

Use balanced accuracy/PR-AUC rather than plain accuracy, class weights, resampling (SMOTE, undersampling), threshold tuning, collecting more minority data.

Q11. How do you serve a Python model with low latency?

Load the model once at startup (not per request), keep inference light (precomputed feature extraction), use FastAPI/Flask behind gunicorn/uvicorn, batch if possible, cache results, and monitor latency.

Edge cases and gotchas

- Python integers are arbitrary precision; floats are IEEE 754.
- Late-binding closures in loops (`lambda: i` all see the last `i`).
- `sorted` returns a new list; `list.sort()` sorts in place and returns `None`.
- Modifying a list while iterating skips elements.
- A model pickled with one scikit-learn version may fail to load in another; pin versions in Docker.
- Random seeds (`random_state`) are needed for reproducibility.

Rapid-fire round

Question	Answer
Python compiled or interpreted	Compiled to bytecode, executed by a VM
`__init__` vs `__new__`	Initialiser vs object creator
`@staticmethod` vs `@classmethod`	No implicit arg vs receives the class
Jinja is	A templating engine rendering HTML from Python data (Flask default)
`pip freeze`	Lists installed packages with versions
Supervised vs unsupervised	Labelled targets vs finding structure without labels

Section 2 — Non-Technical and Resume-Based Questions

How to use this section: the model answers below are built only from facts printed on your resume. Where a question needs details only you know (what you personally did, the exact tools, the real numbers behind a metric), the answer is marked as a frame to fill in. Interviewers punish invented detail much harder than missing detail, so use your real story.

Introduction and HR Questions

> Short, structured answers (60-90 seconds). Use the pattern: present, past, future.

Q1. Tell me about yourself.

Sample frame: "I'm Shreyansh, a third-year B.Tech IT student at MAIT, Delhi, with a GPA of 8.97. I'm a full-stack developer who likes backend and cloud work. I've interned at Procurpal, where I built e-signing integrations and a real-time auction module, and at MyAiTree, where I cut page load from 4 seconds to under 500 ms. Outside work I've built projects like Alethea (AI log intelligence) and BehaveGuard (behavioural authentication) and contributed 14+ patches to Git itself. I'm now looking for a role where I can work on production backend systems and keep learning from strong engineers."

Tip: end with a bridge to the role you are interviewing for.

Q2. Why should we hire you?

Link three things to the job: (1) production exposure: two internships with measurable results; (2) depth beyond coursework: open-source work reviewed by Git maintainers, an ML-based security system with a dataset of 600k+ points; (3) learning speed and ownership: hackathon results and community leadership. Then add one line on how that helps *this* team.

Q3. Why this company / this role?

Research the product, stack and engineering blog beforehand. Connect their technology to something you've built (e.g. "you run on AWS and Node, which I used at Procurpal"). Avoid generic answers like "good culture" and "learning opportunities" on their own.

Q4. What are your strengths?

Pick two with proof: **learning quickly and shipping** (a Git contributor reviewed by the maintainers; moved from intern to owning an integration), and **performance/measurement mindset** (every resume bullet has a number). Support each with one short story.

Q5. What is your weakness?

Choose a real, non-fatal weakness and show the fix. Examples that fit a student engineer: taking on too many projects and hackathons at once (now prioritise and timebox), or going deep on optimisation before validating the need (now measure first). End with what you did about it.

Q6. Tell me about a challenge or failure. (STAR method)

Use **S**ituation, **T**ask, **A**ction, **R**esult (plus what you learned). Good source stories from your resume: a patch revision after tough review feedback on the Git mailing list; a performance problem such as 4000 ms load times; a deployment pipeline that took 10 minutes. Keep Action about *you*, say "I" for your part and "we" for the team.

Q7. Describe a conflict with a teammate or reviewer and how you handled it.

Strong example: mailing-list review where a maintainer rejected your approach. Show: you read the feedback carefully, asked a clarifying question, reworked the patch, thanked the reviewer, and the series was merged. Shows humility and the ability to take critical feedback.

Q8. How do you work under pressure or tight deadlines?

Mention hackathons (3rd place at Hacknovate; top 50 at Smart India Hackathon 2025): scoping an MVP, splitting tasks, daily checkpoints, cutting scope instead of quality. Add how you communicate risks early.

Q9. How do you handle working remotely?

Both internships were remote (Procurpal in Bangalore, MyAiTree in New Delhi). Talk about written updates, clear PR descriptions, asking questions early, overlapping hours, documenting decisions, and using task boards.

Q10. Where do you see yourself in 5 years?

Becoming a strong engineer who owns systems end to end: deepening backend/cloud and distributed systems, mentoring juniors, and contributing to open source. Keep it tied to growth in the role, not a job title.

Q11. How do you keep learning?

Reading source code and RFCs (Git codebase), building side projects to force new concepts (RAG, behavioural biometrics), participating in hackathons, reading engineering blogs, reviewing others' PRs.

Q12. Tell me about a time you led others.

Use your IOSD (Web Dev Head) and TEDxMAIT (Core Web Developer) roles: organising hackathons of 350+ participants and leading full-stack workshops. Give team size, what you owned, a problem you solved (registrations, load, sponsor deadlines) and the outcome.

Q13. How do you prioritise between academics, projects and open source?

Be honest and specific: a weekly plan, fixed blocks for coursework, projects timeboxed, and the GPA (8.97) as evidence that you manage both.

Q14. Do you have any questions for us? (Always say yes.) Good ones:

- What does the onboarding and first 90 days look like for this role?
- How does the team handle code review, testing and on-call?
- What is the hardest technical problem the team is working on now?
- How are interns/new grads mentored, and how is performance evaluated?
- What does the deployment pipeline look like, and how often do you ship?

Edge cases and gotchas

- Don't badmouth previous teams or companies.
- Don't memorise a monologue; know 4-5 bullet points per story.
- Never claim sole credit for team work; interviewers will probe with "what exactly did you do?".
- Be ready for "Why are you leaving?" even for internships: "The internship ended as planned / I wanted exposure to X."
- Salary questions: research the range, answer with a range, and say you are flexible for the right learning opportunity.

Rapid-fire round

Question	Answer
Notice period / availability	Know your graduation date (May 2027) and when you can start
Relocation / remote	Decide beforehand
Biggest achievement	Pick one with a number (4000 ms to 500 ms; 14+ Git patches; 95.6% balanced accuracy)
Role model engineer	Name someone whose work you actually read
Favourite project	Choose the one you can talk about for 10 minutes

Procurpal (Software Engineer Intern, Dec 2024 - Mar 2025)

> Claims to defend: Qsign API integration with TypeScript, Cron Jobs and DynamoDB reducing sync delays by 50%; real-time e-Auction module (React, WebSockets, Tailwind) with 20+ screens; Lambda/Docker pipeline: deploy time 10m to 3.6m, artifact storage 5x smaller.

Q1. Walk me through your role and the Qsign integration.

Frame: "Procurpal needed e-signing inside its document workflow. I owned the backend integration with the Qsign API: designing REST endpoints to create signing requests, track document status and store the results, with TypeScript services, scheduled cron jobs for synchronising signing status, and DynamoDB as the data store. The goal was to automate the workflow and reduce the delay between a document being signed and the platform reflecting it." Add: how many endpoints, who used it, what the user-visible flow was.

Q2. How did you reduce sync delay by 50%? How did you measure it?

Be precise about the mechanism you used: for example shorter or smarter polling intervals, polling only documents in pending states, batching, event/webhook handling, parallel processing, or removing redundant calls. State the baseline (e.g. average time from signature completion to status update before vs after), how you measured (timestamps stored in DynamoDB, logs, CloudWatch), and over what sample. If the number was an estimate, say so.

Q3. Why cron jobs instead of webhooks? What failure cases did you handle?

Good answer shapes: the third-party might not provide reliable webhooks; cron gives a reconciliation guarantee. Handle: overlapping runs, retries with backoff, API rate limits, partial failures (continue other documents), idempotent updates (conditional writes), alerting on repeated failures, and not double-processing when running on several instances.

Q4. Why DynamoDB? What would your data model look like?

Serverless fit with Lambda, no connection pool issues, predictable low latency, pay-per-use, simple access patterns (lookup by document ID, list by status). Describe your partition/sort keys, any GSI (e.g. status + updated time), how you avoided scans, and how you used conditional writes to prevent overwriting newer states.

Q5. What security concerns exist in e-signing integrations?

Signed document integrity and auditability, access control (who can view/sign), signed URLs with expiry for document storage, webhook signature verification, secrets in a secrets manager, encryption in transit and at rest, audit logs, and legal validity depending on the provider.

Q6. Describe the e-Auction module architecture. What was hard?

Frame: React UI with WebSocket connection for live bids, Tailwind-based responsive screens (20+). Hard parts to talk about: consistency of the highest bid under simultaneous bids (server as the source of truth, atomic validation), time synchronisation of the countdown, reconnect and state resync, UI performance under frequent updates, and edge cases (bid at the last second, ties, retracted bids, user disconnected). See Socket.IO topic in Section 1.

Q7. How did you build 20+ screens in a short internship?

Reusable components, a shared design token/Tailwind config, composition of layouts, mock data/API contracts agreed early with backend, and reviewing designs (Figma) for repeated patterns. Be ready to say how many were unique vs variations.

Q8. Explain the Lambda and Docker optimisation (10m to 3.6m, 5x smaller artifacts).

Walk through the baseline pipeline, the bottleneck you found (large dependency install, big images or zip files, rebuilding everything, no caching), what you changed (see the Docker and CI/CD topics for the possible levers), and how you verified (before/after build logs, package size). Mention risks: cold-start impact, missing dependencies at runtime, and how you tested.

Q9. What would you do differently or improve now?

Good, honest answers: add webhooks plus reconciliation, add tests and contract tests for the third-party API, structured logging and alarms, infrastructure as code, load-test the auction under concurrent bids, add idempotency keys.

Q10. How did you work with the team remotely? How were you evaluated?

Standups, ticket workflow, PR reviews, communication channel, who your mentor/manager was, what feedback you received and acted upon.

Edge cases and gotchas

- Interviewers often ask "what percentage of this was your own work?" Prepare an honest answer.
- If you cannot explain a bullet in two minutes, rewrite the resume bullet.
- Do not reveal confidential company data (customer names, internal prices); describe at a general level.

Rapid-fire round

Question	Answer
Duration	Dec 2024 to Mar 2025, remote, Bangalore-based company
Stack	TypeScript, AWS (Lambda, DynamoDB), Docker, React, WebSockets, Tailwind
Key numbers	50% less sync delay; 20+ screens; 10m to 3.6m deployment; 5x smaller artifacts
Biggest learning	Be ready with one specific lesson
Would you reuse DynamoDB	Yes for key-based access; no for ad hoc analytics

MyAiTree (Software Engineer Intern, Jun - Aug 2024)

> Claims to defend: client-side image caching reducing page load from 4000 ms to under 500 ms and improving PDF parsing/rendering; Python computer-vision backend integrated with the frontend through Jinja for real-time inference display.

Q1. How did you cut page load from 4000 ms to under 500 ms?

Describe: how you identified the problem (Chrome DevTools Network/Performance, Lighthouse): many large image/asset requests re-downloaded on each visit. The fix: a client-side caching layer (for example an in-memory map for the session plus persistent storage via Cache API/IndexedDB/service worker, keyed by URL with versioning and expiry), plus proper HTTP cache headers where possible. Explain invalidation (versioned URLs, ETag, TTL), memory limits (size cap/LRU), and how the numbers were measured (cold vs warm cache, which page, which network profile). Be clear whether 500 ms applies to repeat visits.

Q2. What are the trade-offs and risks of client-side caching?

Stale data, storage quotas and eviction, privacy on shared devices, cache poisoning if keys aren't unique, inconsistent behaviour across browsers, and complexity in invalidation ("the hardest problem"). First load stays slow without CDN/optimisation, so combine with image compression, lazy loading and CDN.

Q3. How did caching improve PDF parsing and rendering?

Likely mechanisms: caching fetched PDF bytes and rendered page images/thumbnails, avoiding repeated downloads and re-parsing, rendering lazily per visible page. Mention the library used (for example pdf.js) only if you used it.

Q4. Explain the Python computer-vision backend and how you integrated it with Jinja.

Frame: the AI service (Python, computer-vision model) exposes inference; Jinja templates render the HTML with the results, so users see inference output in the UI in real time. Be able to say: which framework served it (Flask/FastAPI), model input/output, how results were passed to the template, whether real-time meant polling/streaming/sockets, and the latency you observed.

Q5. What is Jinja and what are its risks?

Jinja is a templating engine: variables `{{ x }}`, control flow `{% if %}`, filters, inheritance (`extends`, `block`). It auto-escapes in Flask by default (XSS protection); the main risk is **server-side template injection** when user input is rendered as a template rather than as data.

Q6. How would you make that inference pipeline scale?

Run the model in a separate service/worker, use a queue for heavy jobs, load the model once, batch requests, GPU or optimised runtime (ONNX), cache repeated results, return a job ID and push results by WebSocket/SSE instead of blocking.

Q7. What did you learn from that internship?

Examples: profiling before optimising; working with a team across Python and web layers; the value of measurable results.

Edge cases and gotchas

- Interviewers may ask "what is Time to First Byte vs LCP?" Know basic web vitals (LCP, CLS, INP).
- Be ready to explain what "page load latency" meant exactly (load event? first contentful paint?).
- If you did not train the computer-vision model, say so; focus on integration.

Rapid-fire round

Question	Answer
Duration	Jun to Aug 2024, remote, New Delhi
Key numbers	4000 ms to under 500 ms
Stack	Client-side caching, PDF rendering, Python AI backend, Jinja
Core Web Vitals	LCP (loading), INP (interactivity), CLS (visual stability)

Alethea (Agentic AI log intelligence)

> Claims to defend: Agentic AI-driven pipeline that parses raw logs, detects anomalies, reconstructs failure context, and generates LLM-based incident narratives through a RAG debugging chatbot, reducing incident resolution time by 33%; real-time PR analysis engine using GitHub Octokit and webhooks. Stack: React, TypeScript, Next.js, Llama 3, Node.js, Docker, MongoDB, Redis, GitHub OAuth.

Q1. Explain Alethea in one minute.

Frame: "Alethea ingests raw application logs, parses them into structured events, detects anomalous patterns, reconstructs the context around a failure (the events before and after it, related services), and uses an LLM (Llama 3) to write an incident narrative. A RAG chatbot lets engineers ask questions about incidents and answers using retrieved logs and past incidents. A second component analyses pull requests via GitHub webhooks and flags high-risk changes. Result: 33% faster incident resolution in our evaluation."

Q2. What does "agentic" mean here?

An LLM-driven system that decides which tool to call next in a loop (plan, act, observe) instead of a single prompt: e.g. query logs, fetch the stack trace, search similar past incidents, then write the summary. Be precise about your agent's tools, loop limit, and guardrails, or avoid the word if the pipeline is mostly a fixed workflow.

Q3. How did you parse logs and detect anomalies?

Frame the possible approaches and state yours: regex/grok parsing or log-template extraction (e.g. Drain algorithm), normalising timestamps/levels/services; anomaly detection via statistical thresholds (error-rate spikes, z-scores), rarity of templates, sequence models, or LLM-assisted classification. Mention trade-offs: false positives, alert fatigue, and the importance of baselines.

Q4. How does your RAG chatbot work?

Chunk logs/incident summaries, create embeddings, store them in a vector index (state which one you used), retrieve top-k relevant chunks for the question (often with metadata filters like service and time), put them in the prompt, and have Llama 3 answer with citations to log lines. Improvements: hybrid search (BM25 + vectors), reranking, query rewriting, chunking by time window/trace ID, and evaluation sets.

Q5. How do you reduce hallucinations in incident narratives?

Ground answers in retrieved evidence, force citations to specific log lines, lower temperature, structured output schemas, validate that every cited line exists, say "I don't know" when retrieval is weak, and keep a human in the loop.

Q6. How did you measure the 33% reduction in incident resolution time?

You must be able to defend this. State the baseline (manual debugging time on a set of injected or historical incidents), the sample size, who measured, and what "resolution" meant (time to identify root cause vs time to fix). If it was a small controlled test, say "in our tests" and not "in production".

Q7. How does the PR analysis engine work? (Octokit and webhooks)

GitHub sends a `pull_request` webhook; your server verifies the **HMAC SHA-256 signature** in `X-Hub-Signature-256`, responds quickly (2xx), enqueues the work, fetches the diff with Octokit, evaluates it (heuristics such as files touched, size, sensitive paths, plus LLM review), and posts or stores a risk score. Idempotency via the delivery ID (`X-GitHub-Delivery`); handle API rate limits and large diffs (chunking).

Q8. How did you handle GitHub OAuth and permissions?

Authorization code flow with `state` to prevent CSRF, minimum scopes, tokens stored server-side and encrypted, and repository access limited to what the user granted. Know the difference between an OAuth App and a **GitHub App** (fine-grained permissions, installation tokens, higher rate limits, preferable for integrations).

Q9. Why Redis and MongoDB in this project?

MongoDB for flexible log/incident documents; Redis for caching, rate limiting, and queueing/real-time state. Be ready to justify and to discuss alternatives (Elasticsearch/OpenSearch for logs, Postgres with pgvector, a dedicated vector DB).

Q10. Why Llama 3 and how did you run it?

Open model, self-hostable (privacy: logs may contain sensitive data), no per-token API cost. Explain how you ran it (Ollama, vLLM, hosted endpoint), context window limits, latency and how you handled long logs (summarise, window, retrieve).

Q11. What are the privacy and security issues with sending logs to an LLM?

Logs contain secrets/PII; redact before processing, prefer self-hosted models, restrict access, protect against **prompt injection** (log lines are untrusted text and may contain instructions), and audit what is stored.

Q12. How would you scale ingestion to millions of log lines per minute?

Message broker (Kafka/Kinesis/SQS), stream processing, batching, sampling, partitioning by service/time, tiered storage, and keeping the LLM stage off the hot path (only anomalies reach it).

Edge cases and gotchas

- Expect "what are the limitations?": false positives, LLM cost/latency, evaluation difficulty, hallucination, dependence on log quality.
- Expect "what did you build vs reuse?": be clear about libraries and models.
- Prompt injection through logs/PR content is a favourite follow-up.
- If asked about evaluation, mention precision/recall of anomaly detection and human-rated narrative quality.

Rapid-fire round

Question	Answer
RAG expansion	Retrieval-Augmented Generation
Embedding	Vector representation of text meaning
Webhook signature header	`X-Hub-Signature-256`
Key numbers	33% faster incident resolution
OAuth vs authentication	OAuth is delegated authorisation; login uses it plus identity (OIDC)
Temperature	Controls randomness of LLM output

BehaveGuard (Behavioural authentication)

Claims to defend: keystroke and mouse-dynamics authentication system derived from the CMU Benchmark (2009); feature extraction pipeline deployed for real-time verification; longitudinal multi-user dataset of 600k+ data points; 95.6% balanced accuracy and 2.1% EER; Docker + GitHub Actions to Google Cloud Run; PostgreSQL for enrolment/verification; Redis for sessions and rate limiting.

Q1. What is BehaveGuard and why is it useful?

A behavioural biometric system: it verifies who is typing or using the mouse from *how* they do it (timings and movement patterns) rather than what they know. It adds continuous, passive authentication on top of passwords, helping against stolen credentials and session hijacking, without extra user friction.

Q2. What is the CMU benchmark?

The 2009 study by Killourhy and Maxion, "Comparing Anomaly-Detection Algorithms for Keystroke Dynamics", which released a public dataset of keystroke timings from 51 subjects typing the same password many times, and compared many detectors (Manhattan, Mahalanobis, nearest neighbour, SVM and others) under one evaluation protocol. It's the standard benchmark for keystroke dynamics research. Know exactly how *your* work used it (features, protocol, baseline) versus your own collected dataset.

Q3. Which features did you extract?

Keystroke: **dwell/hold time** (key down to key up), **flight time** (key up to next key down), **down-down latency**, digraph timings, typing speed, error/backspace rates. Mouse: velocity, acceleration, curvature, click durations, pauses, movement angles, straightness. State the ones *you* used and how you normalised for different devices.

Q4. Explain EER, FAR, FRR and balanced accuracy. What do your numbers mean?

FAR: impostors wrongly accepted; **FRR**: genuine users wrongly rejected. Moving the decision threshold trades one against the other. **EER** is the point where FAR = FRR; 2.1% means that at that threshold about 2.1% of impostors are accepted and 2.1% of genuine attempts are rejected. **Balanced accuracy** is the mean of recall on each class (genuine and impostor) at a chosen operating threshold. Be ready for: "an EER of 2.1% implies roughly 97.9% accuracy at that point, so why is balanced accuracy 95.6%?" Answer: they are measured at different thresholds and potentially different protocols/splits; balanced accuracy was computed at the deployed threshold, not at EER. Confirm this against your own evaluation before the interview.

Q5. Which model did you use and why?

State the actual model (e.g. a scikit-learn classifier or anomaly detector) and the reasoning: small feature vectors, need for low-latency inference, interpretability, per-user models vs one global model, one-class (only genuine data) vs binary (genuine plus impostors). Mention hyperparameter tuning and how you compared with baselines.

Q6. How did you collect the 600k+ data points and avoid bias?

Describe the collection process (volunteers, sessions over days/weeks, task design, devices), consent and anonymisation, cleaning (outlier removal, filtering sessions), and class balance. A **longitudinal** dataset lets you test *temporal drift*: behaviour changes over days. Explain the train/test split: by user/session and time, not random rows, to prevent leakage.

Q7. How do you handle concept drift and cold start?

Drift: periodically re-enrol or adaptively update templates (with safeguards against poisoning), use recent windows, monitor FRR trends. Cold start: require N enrolment samples before enforcing, fall back to password/OTP and step-up authentication when confidence is low.

Q8. What attacks and limitations exist?

Impersonation/mimicry attacks, replay of recorded timings, bots simulating input, injected synthetic events, user variability (injury, fatigue, different keyboard/device), privacy of behavioural data, and fairness for users with disabilities. Mitigation: use as one factor in risk-based authentication, liveness/anti-replay checks, rate limits, and encrypted storage.

Q9. Describe the architecture and real-time verification path.

Client captures events (timestamps only, not the characters typed, where possible), sends a feature batch to the API; the service computes features, loads the user's model/profile, returns a score and decision; PostgreSQL stores enrolment and verification records, Redis keeps session state and enforces rate limits; the container runs on Cloud Run, built and deployed by GitHub Actions. Be ready with latency numbers.

Q10. Why Redis for sessions and rate limiting here? Why Cloud Run?

Cloud Run instances are stateless and scale out, so session/rate-limit state must be shared: Redis. Rate limiting protects verification endpoints from brute-force and probing attacks. Cloud Run gives autoscaling and per-request billing with zero server management. See Redis and Cloud Run topics in Section 1.

Q11. What privacy and legal issues arise?

Behavioural biometrics are personal data: obtain consent, minimise what you store (timings rather than raw keystrokes), encrypt, define retention and deletion, and comply with regulations (e.g. India's DPDP Act, GDPR if relevant). Never log typed content.

Q12. How would you improve BehaveGuard?

Per-user adaptive thresholds, continuous authentication, deep models (LSTM/transformer on event sequences), cross-device normalisation, federated or on-device learning for privacy, red-team evaluation with attacker datasets, monitoring dashboards, A/B testing.

Edge cases and gotchas

- Expect "show me the confusion matrix" thinking: be able to compute precision/recall from numbers.
- "Is 95.6% good?" depends on the threshold, population size and attack model; do not oversell.
- Be ready to explain why accuracy alone is misleading with imbalanced classes.
- You said "co-architected": be clear which parts you personally built.

Rapid-fire round

Question	Answer
EER at a glance	Threshold where FAR equals FRR; lower is better
Dwell vs flight time	Key hold duration vs gap between keys
Key numbers	600k+ data points; 95.6% balanced accuracy; 2.1% EER
Benchmark	CMU keystroke dynamics dataset, 2009
Stack	Python, scikit-learn, PostgreSQL, Redis, Docker, GitHub Actions, Cloud Run
Why not accuracy	Class imbalance and threshold dependence

Open Source (Git, Augtoberfest, OpenNyAI)

> Claims to defend: 14+ patches merged into the Git project via the kernel.org mailing list, in v2.54.0; Top 10 of 5,000+ in Augtoberfest (Code4GovTech) with a merged PR in OpenNyAI.

Q1. How did you get started contributing to Git? What did you work on?

Prepare a clear story: how you found the project (a microproject / "good first issue" / #leftoverbits / newcomer guidance), your first patch and what it fixed, how review went, and how contributions grew to 14+ merged patches. Name the areas (for example a builtin command, a test cleanup, documentation, refactors), the language (C and shell tests), and the version where they shipped.

Q2. Walk me through one patch in detail.

Structure: problem (what was wrong or inconsistent), investigation (reading code, reproducing, tests), solution (what you changed and why this approach), review (what the maintainers or reviewers asked, what you changed in v2/v3), result (merged into which release). Have 2 or 3 of these ready, including commit hashes or titles.

Q3. What did you learn from the mailing-list review process?

Small focused patches, clear commit messages (explain *why*, imperative mood), tests with every behaviour change, patience with asynchronous review, responding to every comment, respecting maintainers' time, no "drive-by" noise, and how to disagree politely with evidence. See the Git topic in Section 1 for the workflow vocabulary.

Q4. How is that different from the GitHub pull request flow?

Mailing list: patches in email, inline review in replies, no central web UI, versioned series (v2, v3) with changelogs, maintainer collects topics into integration branches. GitHub PRs: web UI, review comments on diffs, CI checks, merge buttons. The list is slower to start but teaches rigorous communication.

Q5. How did you handle disagreement or rejected feedback?

Show an actual example where you changed your approach, or respectfully explained your reasoning with data and the reviewer agreed.

Q6. Tell me about Augtoberfest and OpenNyAI.

Augtoberfest (Code4GovTech) was a month-long open-source contribution programme; you ranked in the top 10 out of 5,000+ participants, with a merged PR in **OpenNyAI**, an open-source AI-for-Justice initiative (use of AI for legal access in India). Know what your PR did, the tech involved, how the ranking worked (points per merged PR/issue), and what you learned about contributing to a large unfamiliar codebase.

Q7. How do you approach contributing to a codebase you don't know?

Read README/CONTRIBUTING, build and run tests, find a small issue, trace the code with grep/debugger, ask a precise question after investigating, propose a minimal change with tests, and follow the project's style.

Q8. What do you look for when reviewing someone else's code?

Correctness, readability, tests, edge cases, security, performance only where it matters, consistency with the codebase, scope of the change, and kindness in tone.

Edge cases and gotchas

- If asked to open the Git codebase live, be able to navigate `builtin/`, `t/`, `Documentation/`.
- Know the difference between your merged patches count and your commits (a patch series may have several commits).
- Do not exaggerate: "contributed to Git" is impressive enough when concrete.

Rapid-fire round

Question	Answer
Where Git patches go	The Git mailing list (git@vger.kernel.org)
Sign-off meaning	Developer Certificate of Origin: you have the right to submit this code
Tool to send patches	`git send-email` (or GitGitGadget)
Series versions	`[PATCH v2]`, `[PATCH v3]` with a changelog
Integration branches	`seen`, `next`, `master`, `maint`
Programme	Augtoberfest by Code4GovTech; OpenNyAI

Education, Hackathons and Leadership

Claims to defend: B.Tech IT at GGSIPU (MAIT), GPA 8.97/10, 2023-2027; 3rd place at Hacknovate (Sustainable Development); top 50 teams at Smart India Hackathon 2025; Web Dev Head at IOSD; Core Web Developer at TEDxMAIT; hackathons with 350+ participants; full-stack workshops.

Q1. Tell me about your Hacknovate project (3rd place).

Be ready with: the problem statement (sustainable development theme), your solution, stack, team size, your role, a technical decision you made, what the judges liked, and what you would improve. Have a 90-second demo narrative.

Q2. Tell me about Smart India Hackathon 2025.

Explain the problem statement, how many teams you competed with, the internal and national rounds, how you were shortlisted into the Top 50, your solution's architecture, and feedback from the evaluators. Mention what you learned from presenting to the jury.

Q3. What did you do as Web Dev Head at IOSD?

Lead the team building the community's websites/tools, assign tasks, review code, mentor juniors, plan workshops. Give numbers: team size, number of workshops, attendees, projects shipped, a conflict or deadline you handled.

Q4. How did you help organise a hackathon with 350+ participants?

Describe your part: registration/website, judging and scoring tools, infrastructure, communication, problem statements, mentors, sponsors, logistics, and issues you solved live (traffic spikes, duplicate registrations). Highlight scale lessons.

Q5. What did you teach in the full-stack workshops and how did you structure them?

Topic breakdown (HTML/CSS/JS, React, Node/Express, DB, deployment), hands-on exercises, a final mini-project, feedback collected, and how you adapted to beginners.

Q6. Why did you choose Information Technology? How does your coursework relate to your work?

Link subjects (DBMS, OS, networks, DSA, software engineering) to what you practise: PostgreSQL/Mongo schema design, process/thread concepts behind Node and Docker, TCP/HTTP behind WebSockets, etc.

Q7. How do you balance GPA with projects?

Consistent weekly schedule, picking projects that overlap with coursework, saying no to low-value commitments. Your GPA of 8.97 shows you do not trade academics for projects.

Q8. What is a technology you'd like to learn next and why?

Choose something relevant to the role, e.g. Kubernetes and observability for production systems, or distributed systems, or LLM evaluation. Say how you'd start (a small project, docs, a talk).

Edge cases and gotchas

- Know the full forms: IOSD (the college developer community name as written on your resume), TEDxMAIT, SIH (Smart India Hackathon).
- Hackathon questions often become system design questions; be ready to whiteboard your solution.
- Be ready for CS fundamentals (OS, DBMS, networks) since you list education prominently; they are not in this module.

Rapid-fire round

Question	Answer
Degree and years	B.Tech IT, Sept 2023 to May 2027, GGSIPU (MAIT), New Delhi
GPA	8.97 / 10.00
Hackathon results	3rd at Hacknovate; Top 50 at SIH 2025
Leadership	Web Dev Head at IOSD; Core Web Developer at TEDxMAIT
Scale of events	350+ participants

Resume Number Cheat Sheet and Red-Flag Questions

> Memorise every figure and be able to explain how you got it.

Q1. Which resume numbers will I be asked to justify?

Each number is a likely probe. Prepare: baseline, method, sample, and caveat.

- Procurpal: sync delays reduced by 50%; 20+ production-ready screens; deployment 10 min to 3.6 min; artifact storage 5x smaller.
- MyAiTree: page load 4000 ms to under 500 ms.
- Alethea: incident resolution time reduced by 33%.
- BehaveGuard: 600k+ data points; 95.6% balanced accuracy; 2.1% EER.
- Open source: 14+ patches merged; released in Git v2.54.0; Top 10 of 5,000+ in Augtoberfest.
- Extras: GPA 8.97/10; Hacknovate 3rd place; Top 50 teams at SIH 2025; events with 350+ participants.

Q2. "This sounds impressive, but what exactly did YOU do?"

Answer with specifics: name the files/modules/endpoints you wrote, the decisions you made and one thing that went wrong. Use "I" for your part and "we" for the team. Offer to walk through code or a diagram.

Q3. "How do you know that number is real?"

Explain the measurement: tool, dataset or time window, before/after comparison. Admit limits: "this was measured in staging / on a test set of N cases". Honest caveats build trust.

Q4. "What is the weakest part of your resume?"

Pick something true and show awareness, e.g. limited production scale experience (internships were short), then explain how you compensate: open-source rigour, side projects, learning plan.

Q5. "Which skill on your resume are you weakest at?"

Name one sincerely (e.g. system design at scale, or Kubernetes) and say what you are doing about it. Never claim equal expertise in everything on the Technical Skills list.

Q6. "Teach me something technical in two minutes."

Prepare one concept you know deeply (event loop, DynamoDB partition keys, Git internals, EER and FAR). Use an analogy, a small example and a pitfall.

Edge cases and gotchas

- Anything on your resume is fair game, including tools listed under Technologies you used lightly: be honest about depth ("used in a project, comfortable with basics").
- Interviewers cross-check across rounds; keep your story consistent.
- Have the project repos/websites (GitHub, personal site) clean and working; they will open them.
- Dates: Procurpal Dec 2024 to Mar 2025; MyAiTree Jun to Aug 2024.

Rapid-fire round

Question	Answer
Longest project you can discuss	Pick one (Alethea or BehaveGuard) and prepare a 10-minute deep dive
Proudest line	Pick one with a number and a story
Most technical challenge	Real-time bidding consistency, or drift in behavioural data

Question	Answer
If you had a week more	Tests, monitoring, load testing, docs